

P?S/8 Keyboard Monitor/Editor Commands and Additional Information.

Copyright © 1975-1989 Charles Lasner Associates

Copyright © 1990-2016 »CLA Systems

CLASystems@gmail.com

Last edit: 28-Jun-2016 cjl

Document preparation was performed using the excellent freeware word-processing program **Jarte** developed by Carolina Road Software, LLC. Please be advised that attempts to use various Microsoft products will destroy key line formatting and is not recommended.

This document is a guide to the P?S/8 keyboard monitor and integrated line editor. All available commands are documented sufficiently for beginning users to get started using P?S/8 for common text editing, initiating program execution and related tasks.

Conventions used in this document.

All commands are given in upper-case characters; as an accommodation, lower-case characters are "folded" to the upper-case equivalent as typed. As such, the commands are shown in all uppercase. With the exception of the R command, all commands require the first two characters as shown; additional characters are allowed and will be ignored. Commands will be shown with certain optional trailing characters enclosed in braces indicating the usual meaning of the command; these are provided for clarification purposes only.

Commands that access files by name.

Certain commands allow optional arguments that are file names taken from the standard P?S/8 directory structure known as the Tiny File System [TFS]. Where this usage applies, the command will be followed by *filename*. [Note: The actual TFS file names are up to six upper-case characters.]

In general, file-oriented commands can also reference the special positional multiple-purpose files known as % and \$ [which are located near the physical start of every system device unit for faster access].

Logical device unit access.

Many commands indicate an optional *:u* suffix. The current logical unit in effect can be updated using an appended :0 through :7 corresponding to the desired logical device unit.

In general, commands that allow the optional *:u* suffix and optional file name arguments will allow the *:u* suffix on the passed file names. [Note: Only commands for program execution allow multiple file names to be specified; the optional *:u* suffix is allowed on the program name and every specified input or output file name as required.] All commands and file name options will indicate the optional *:u* suffix where applicable.

Logical unit interpretation.

Logical units are processed by the system device handler in a hardware-dependent manner. Representative system configurations are described below:

1) TC01/TC08/TU55/TU56 DECTapes.

The logical units refer to the physical DECTape drive tape units which can be changed using drive-mounted unit-select switches. The system device handler uses the current logical unit directly.

2) TC12/TU55/TU56 LINCTapes.

The logical units refer to the physical LINCTape drive tape units which can be changed by using drive-mounted unit-select switches. The system device handler uses the logical unit in effect directly; this is analogous to the TC01/TC08/TU55/TU56 system.

3) TD8E/TU56 DECTapes.

The low-order bit of the logical unit is the only information used by the system device handler to select the physical drive. Thus, all references are to either drive 0 [for all even logical units] and drive 1 [for all odd logical units]. The physical DECTape drive tape units can be selected as in the TC01/TC08/TU55/TU56 system; however, only drives 0 and 1 can be accessed.

There are two variations of TD8E system support:

- a) The system requires the MR8E-C ROM that occupies locations 77400-77777. This system will run with 4K of read/write memory in field 0; the largest configuration is 28K [due to the presence of the ROM in field 7].

Note: System device handler support for additional drive units is not possible for the MR8E-C ROM-based system due to internal coding restrictions within the ROM itself.

- b) The system requires 8K of memory in field 0 and field x where x is the highest available field [1 through 7]. Any unused extended memory [if more than 8K is present] is available without restriction.

Note: Future versions of the TD8E system device handler may support up to four TD8E controllers. Each TD8E adds two additional drive units [0-1 for the primary controller, 2-3 for the second controller, 4-5 for the third controller and 6-7 for the fourth controller]. In such a configuration, the system device handler will use the logical unit in effect directly. This configuration would be analogous to the TC01/TC08/TU55/TU56 system with the restriction that each TU56 can only select one designated drive pair.

4) RX01/RX8E diskettes.

The basic RX01 configuration is a single pair of 8" diskette drives referred to as logical unit 0 and 1. For this form of system configuration, the low-order bit of the logical unit is the only information used to select the physical drive. As in the case of the TD8E/MR8E-C configuration all even logical units refer to drive 0 and all odd logical units refer to drive 1.

Supported drives include the RX01/RX02/RX03, the two-drive variant of the DSD-210, the DSD-440 and the two-drive variant of any VT78, DECmate I and DECmate II RX278 option board systems.

Some of these configurations support up to four drives such as the VT78, DECmate I, DECmate II RX278 with two RX01/RX02/RX03 drive pairs and the four-drive variant of the DSD-210. On these systems, the two low-order bits of the logical unit are used to select the desired drive unit. The high-order bit of the logical unit is ignored; for example, all references to logical unit 7 are identical to references to drive unit 3.

5) RK8E/RK05 and other mapped disk drives.

On these systems, a portion of a disk [or perhaps the entire disk drive] maps to a logical unit. Depending on exact configuration, the RK8E/RK05-based system may contain all eight drives on a single disk drive. Certain configurations support logical units 0-3 on one physical drive and logical units 4-7 on another drive. Systems configured for four drive support will address two logical drive units on each of up to four RK05 drives. The largest configuration can address each physical RK05 drive as its own logical unit. [Note: Future support could include the RKS8E controller to support up to eight drives.]

On all variations of system configuration, mapped systems support each logical drive unit uniquely [as is the case in the TC01/TC08/TU55/TU56 system, except certain mapped logical units may reside on the same physical disk and cannot be separately dismounted].

Several additional devices are supported by P?S/8. Logical drive unit and other information can be obtained from documents specific to the system device handler that applies.

Commands that reference line numbers.

Certain commands operate on text specified by line number within the edit buffer of the keyboard monitor and integrated editor. Until the contents of the edit buffer are saved to a named file on the system storage device, the edited text is merely a tentative file image within the edit buffer. If a file loaded into the edit buffer is modified, all changes are tentative until the edit buffer is written to a file on the storage device. While line numbers are permanently assigned, various commands are available to change the line number values within the edit buffer; writing to a permanent file updates all file contents including the line numbers.

Command parameters expressed as a range of line numbers.

Certain commands use an optional range of line numbers [indicated in the command description as *line-number-range*] that determine the scope of the command operation:

- 1) nnnn The stated line nnnn is selected; if this line does not exist, no operation occurs.

One such command is the LI[st] command, which outputs the line number and contents of specified lines to the system console terminal. For example, `LIST 1000` outputs line 1000 [if line 1000 exists].
- 2) ssss- The operating range for the command is line ssss through the end of the file. Line ssss need not exist and serves only as a limiting value. If no lines exist within this range, no operation occurs. For example, `LIST 500-` outputs lines 500 and higher through the end of the file.
- 3) -ffff The operating range for the command is the beginning of the file through line ffff. Line ffff need not exist and serves only as a limiting value. If no lines exist within this range, no operation occurs. For example, `LIST -2000` outputs all lines in the file through line 2000.
- 4) ssss-ffff The operating range for the command is lines ssss through ffff. Lines ssss and ffff need not exist and serve merely as limiting values. If no lines exist within this range, no operation occurs. For example, `LIST 300-700` outputs all lines that fall in the range of 300 through 700.
- 5) <none> If *line-number-range* is omitted, the operating range for the command is all lines within the file. If the file is empty, no operation occurs. For example, `LIST` outputs all lines in the file.

Command parameters expressed as a starting line number and line number increment.

Certain commands take an optional line number parameter [and perhaps an optional line number increment]. This is indicated in the command description as *optional-starting-line-number* and *optional-line-increment*. If neither parameter is present, the default starting line number is 100. The default line increment is 10 and can only be changed if the starting line number parameter is present.

For example, the RE{sequence} command reorders the line numbers in a file without changing the textual contents of the lines. The command RE [without parameters] will reorder all line numbers starting at 100; subsequent lines are incremented by 10. RE 1000 50 will reorder all line numbers starting at 1000; all subsequent lines are incremented by 50. The command RE 2000 will reorder all line numbers starting at 2000; all subsequent lines are incremented by 10.

Command parameters expressed by additional means.

Certain commands use other methods for stating parameters that are unique to each command; as required, documentation will be provided within each command description including specific examples.

In the interest of overall brevity, certain command descriptions may reference other commands with expanded description elsewhere; this should always be clearly understood due to the format of the command that applies; it may be necessary to first reference the description of other command[s] to better understand the current command description. [This is unavoidable as many of the commands are inter-related.]

For example, the RE{sequence} command documentation refers to the documentation of the LA{st} command due to the way file resequencing is implemented [which includes an automatic equivalent of the LA{st} command].

History of the P?S/8 command processor and fundamental design issues.

P?S/8 is a direct outgrowth of the predecessor operating system known as the R-L Monitor System [named for the principal authors' initials] or MS/8 [the name used when it was submitted to DECUS]. This more primitive system uses the basic concept of commands with only the first two characters significant, allowing additional optional characters that are ignored [Note: while the R-L Monitor System allows six character file names, system program names are also limited to two characters; additional program name characters are ignored.]

The R-L Monitor System was cosmetically modified [the names of certain commands were changed using trivial binary patches while retaining the command functionality] as a stopgap measure incorporated into the early release of DIBOL-8. This temporary official DEC product, which is similar at the command level to future releases of systems supporting the DIBOL-8 compiler, had only the modest goals of supporting the earliest DIBOL-8 compiler and the ability to load the binary output of compilation.

Note: All of the systems described here use the same binary output format internally, thus identical loading techniques apply. The BIN system utility program exists throughout DIBOL-8, the R-L Monitor System and all releases of P?S/8 albeit with slightly different command syntax due to the evolving nature of the command execution process within each system. P?S/8 adds many additional features and operating modes to the BIN utility; when only file names are passed to BIN, the functionality is identical to The R-L Monitor System and DIBOL-8. Binary files from these systems can be copied to P?S/8 using available commands and utilities. The R-L Monitor System implementation of BIN is limited to 4K memory support only; the binary file format supports 32K memory as implemented only in P?S/8.

The largest weakness of DIBOL-8 [which would be overcome in later releases which became known as COS-300/310 or Commercial Operating System Version 3] was the primitive nature of initiating program execution of the compiler and related utility programs; the only information that can be passed to any system program are the contents of user-specified input files [binary output is directed to an implied portion of the TC01/TC08/TU55/TU56 DECTape, the only device supported in the original DIBOL-8] and a 12-bit octal value expressed on the command-line as =xxxx where xxxx is any 4-digit octal number. [Note: For backward compatibility, P?S/8 internally supports the =xxxx parameter identically to the predecessor systems.]

P?S/8 and COS-300/310 were developed independently but on a parallel track to improve execution processing by implementing a compact form of what is known in other DEC systems as **Concise Command Language** [CCL]. In P?S/8, this includes the ability to pass input and output files and option switches. The =xxxx value, originally only used to specify a starting address to the binary loader, is now used by several programs for various purposes.

P?S/8 initially retained consistency with the R-L Monitor System command set where possible. As a result, a conflict arose between some system program names and the command names. [Note: As described above, the R-L Monitor System only allows two characters for system program names as well as command names. Both P?S/8 and COS-300/310 allow up to six character system program names where additional characters [for program names that are exactly six characters] are ignored which is analogous to the way command names are ignored beyond the first two characters in all of these systems.]

The DIBOL-8 system command names were chosen to allow the execution of the compiler and other projected system utilities avoiding anomaly between these program names and command names [regarding the first two characters]. Execution can always be initiated by execution commands [R or RU{n}] to resolve any conflict that could arise; however, the goal was to avoid the need to routinely resort to this for all common system programs [as well as eliminate certain forms of overhead which is quite noticeable on tape-based systems].

Eventually, a parallel design change was gradually made to P?S/8 over the course of several releases resulting in total consistency between P?S/8 and COS-300/310. For example, the original R-L Monitor System PA{use} command is especially useful during BATCH operations:

.PAUSE TELL OPERATOR TO SETUP PRINTER THEN PRESS ANY KEY TO CONTINUE

The BATCH process is paused; the entire command line is shown on the system console [even if echoing of commands is suppressed]. Execution continues after any key is pressed on the system console keyboard.

As originally implemented in DIBOL-8 [and retained in the later releases of COS-300/310], the P?S/8 command name for this function was changed to PL{ease}:

.PLEASE SETUP THE PRINTER THEN PRESS ANY KEY TO CONTINUE

The general design reasoning for this change was to allow concise command execution of an implementation of the PAL assembler [which is generally found in all of these systems, although the specific assembler program referenced may be a completely different implementation of the PDP-8 Assembly Language utility]. By avoidance of a command name expressed as PA{use}, the assembler program can be initiated by the PAL program name, thus avoiding the need to use R PAL [or perhaps RU{n} PAL] and the attendant overhead.

Known surviving implementations of P?S/8 were created circa 1976 where the command names were in transition [without direct contact with the DIBOL-8 or COS-300/310 development process]. The command names used are somewhat different from the corresponding commands of the R-L Monitor System; however, this release is not consistent with the command names of later releases such as Version 8Z [created circa 1989]. This newer release is being made available as part of an ongoing development process adding additional internal features in preparation for new features such as the P?S/8 SHELL overlay.

Abort character and other control character considerations.

In general, all of the operating systems described here support the **Control-C** character as a general abort and return to the keyboard monitor [by reboot if the abort is detected within a conforming program that will exit to location 07600]. Within the keyboard monitor, pressing **Control-C** will generally abort an incomplete command being typed [perhaps in auto-sequenced line number input mode]. The original character to accomplish this was **Control-D** as used in the R-L Monitor System and early P?S/8 releases. In newer releases of P?S/8, the keyboard monitor abort character was changed to **Control-Z** to be more consistent with other DEC operating systems.

As first implemented in the R-L Monitor System, when the keyboard monitor is accepting input [and subject to abort or other terminal control characters], it is possible to press **Control-R** or **Control-T**. The disposition of these characters is device-dependent. On the TC01/TC08/TU55/TU56 DEctape system, **Control-R** will rewind the current logical unit allowing tape motion to overlap with further command input; the tape will stop in the reverse end zone automatically. Pressing **Control-T** will rewind the tape, then the tape will be forced off the takeup reel. If the selected drive is type TU55, the brakes will be locked; the keyboard monitor resumes control after the tape is fully unloaded.

Current command set issues.

As of P?S/8 Version 8Y, all commands are completely consistent with the corresponding COS-300/310 command set. Many of the utilities planned over the course of time are available in this release with the intended program names. For example:

Both COS-300/310 and P?S/8 support an integrated line editor within the command processor. In both systems, the LN{umber} command is used to automatically number text input; the former command for this purpose was the SE{quence} command. By making this change, both systems support a SET utility to adjust system parameters [without resorting to using the R or RU{n} command]. [Note: In the 1976 P?S/8 Version 8P release, the forerunner utility is somewhat less advanced and known as the VARY program to avoid command anomaly.]

Appendix B covers all changes known to have been considered [whether implemented or not] for the P?S/8 keyboard monitor and integrated line editor command set. It is conceivable some of the specific issues raised might be revisited in future P?S/8 releases.

Command documentation release-related issues.

Due to continued interest in older releases of P?S/8, description of the 1976 P?S/8 command set is included; description for the commands of P?S/8 Version 8Z follows. [Note: Little or no further command name changes are contemplated; all additional work will be directed towards changes to either new or existing system utility programs or in preparation for the optional P?S/8 SHELL overlay currently under development; the P?S/8 SHELL uses a completely different command processor to implement alternate commands consistent with other DEC operating systems found primarily on other computer models.]

P?S/8 release information can be obtained as follows:

- 1) The 1976 release is P?S/8 Version 8P; there are no specific utilities to determine this; P?S/8 developers can derive earlier release versions by inspection of various internal storage locations in a designated system block.

In Version 8P [and all earlier Version 8x releases], the version is stored in relative location 12 of block 0001 as two six-bit characters. This convention was changed in Version 8U to accommodate newer internal bootup procedures required by certain newly-added system devices. [Note: The only known surviving Version 8P system is for the TC01/TC08/TU55/TU56 DECTape.]

- 2) As of Version 8U, the version is now stored in relative location 20 of block 0001 in the same format. The reason for the change is that relative location 12 is loaded into location 00012 and is now used as a preloaded auto-index register during certain internal bootup routines; location 00020 is the first available location beyond the auto-index registers. Other than permanently storing the system version, this location has no permanent contents requirement.

Note: Internal knowledge of P?S/8 is no longer required to obtain version information. Newer versions include the SYSTAT{us} utility which provides overall system version and certain additional system-wide configuration settings.

As of this writing, the currently available release is Version 8Z including several binary patches applied by Charles Lasner, lead program designer of P?S/8. Known bugs have been corrected pending source code cleanup and a new system build to Version 9A.

A list of all available system programs can be obtained as follows:

- 1) For any release through Version 8X:

.R SYSTYP

- 2) For Version 8Y and newer:

.[R] DIRECT{ory} /S [add /T for output to the system console]

P?S/8 1976 [Version 8P] commands.

CA{talog}:*u*

Obtain a quick TFS directory listing on the system console printer.

Note: No storage device access is required unless the optional *:u* is used; this forces the TFS directory to be read into memory from the stated logical device unit; the TFS directory is entirely cached during all keyboard monitor operations.

The output format is four file names per line. Since the maximum number of files allowed is 84, the largest possible printout is 21 lines.

Note: Allocation of storage for files and system programs varies. The P?S/8 Version 8P release for TC01/TC08/TU55/TU56, as used with standard-length DEctapes, allows up to 72 files. Longer DEctapes will allow additional files up to the maximum of 84 files.

Files are displayed in physical order of storage on the current logical device unit. Empty entries show as `<FREE>` and will always be indicated unless physically last in the directory [and thus the first of an indefinite number of inactive directory slots not shown].

Note: Multiple EN{ter} and DE{lete} command operations can lead to displaying a series of trailing empty entries in unusual circumstances. The CA{talog} command will display deleted file entries as empty entries; only the very last entry [or the first entry never used] is completely empty in the sense of either having never been used, or cleared as a special case handled by the DE{lete} command. Trailing completely empty entries are not displayed. For more information regarding empty directory entries, see the description of the DE{lete} command.

DE{lete}:*u filename:u*

Delete a file from the TFS directory. The affected file name is removed from the TFS directory; the underlying file contents are not affected, making it possible to restore the file if the deletion was unintended. See Note 1.

Note: When a file is deleted from the TFS directory, the free directory slot is available for storage of another file; if this should occur, the original file is lost.

P?S/8 1976 [version 8P] commands [continued].

Note: The DE{lete} command does not check to potentially clear the previous entry after clearing what becomes the first of any final completely empty entries. This is a deliberate design decision to aid in file recovery; deleted entries generally contain partial file names [unless the file name was only one or two characters]. Appendix A contains information on recovering from accidental file deletion using keyboard monitor commands and specific system program utilities [if available].

EC{ho} [optional 0 or 1]

Inverts the state of output of command characters if no argument given. EC{ho} 0 forces all output off; EC{ho} 1 forces all output on. Pressing **Control-O** during any output operation in progress overrides this by immediately reversing the echo state for each **Control-O** pressed. This command is generally used within BATCH control files to limit output to the system console.

ED{it} [pre-existing line number]

ED{it} the contents of a specified line in the edit buffer. The ED{it} command is operated through the use of certain control characters unique to the ED{it} command [as well as most of the control characters that are used throughout the command processor].

The operation of the ED{it} command is patterned after the FOCAL, 1969 MO{dify} command with accommodations to the P?S/8 editing environment; this includes features that apply to several operating modes [including the automatic placement of line numbers while entering text]. All control functions operate consistently regardless of mode; additional control characters also apply exclusively to the ED{it} command. Description of the applicable control characters follows:

- 1) within the P?S/8 ED{it} command and the FOCAL, 1969 MO{dify} command, pressing **Control-G** initiates the search mode; the next character typed is searched for from the current cursor position until either a matching character is found or the end of the line is reached.

In FOCAL, 1969, the MO{dify} operation starts as if **Control-G** were already pressed; the P?S/8 ED{it} command starts in character insertion mode after the line number and a **<HT>** character are printed on the system console.

Note: For more information on the exact printing of the **<HT>** character, see the description of the TA{b} command.

P?S/8 1976 [version 8P] commands [continued].

At any point during the ED{it} command operation, all keyboard monitor functions are allowed including the insertion and deletion of text characters. This can permit direct editing of the line number characters [and trailing <HT> separator character] if desired. It is possible to create alternate line numbers with contents based on the specified line number without changing the contents of the specified line. With the line number deleted, it is possible to create keyboard monitor commands to be executed in place based on the contents of the original line; this facilitates using complex commands saved within text files avoiding the need for manual entry for each usage.

Note: It is also possible to create files containing a single complex command; these files can be used with P?S/8 BATCH without any form of editing. By editing out all other lines in a file containing multiple complex commands, a temporary file can be created containing only the desired line; depending on the nature of the command, it may be appropriate to save the file as one of the positional files \$ or % due to their shorter access time.

In FOCAL, 1969, the MO{dify} command will completely process and close out the specified line if the search does not match; when this is unexpected, it can be quite disruptive to the editing process. Additional MO{dify} or other commands will be needed to resolve unforeseen issues.

In the P?S/8 ED{it} command, a failing search merely processes and outputs all the rest of the characters in the specified line [if any] and waits for further input [which could include various abort or restart operations thus preserving the original contents of the specified line].

- 2) In FOCAL, 1969, **Control-L** is used to search for the same character as used in previous searches. In the ED{it} command, **Control-A** is used to search for the same character as used in previous searches. Note: If no prior searches have been performed, **Control-A** will output the rest of the line contents [if any] to the system console terminal and wait for further input. See Note 2.
- 3) In FOCAL, 1969, **Control-J** [<LF>] processes and prints out all remaining characters in the line on the system console terminal; additional editing can be performed ending with **Control-M** [<CR>] to close out the line. See Note 2.

within the ED{it} command, **Control-J** [**<LF>**] will output **<CR>** and **<LF>** and then the current tentative line contents to the system console terminal. This is especially useful throughout the keyboard monitor when using hard-copy console terminals which generally require the display of deleted characters [which can be confusing].

- 4) **Control-U** will completely restart the Ed{it} operation; this includes the printing of the opening line number and **<HT>** characters on the system console terminal. No changes have been made to the specified line.
- 5) When FOCAL, 1969 was designed, the displayed character associated with ASCII code 95 decimal [137 octal] [5F hex] was implemented on early Teletype Model 33 and similar console terminals as **␣**; the displayed character was later phased out of the ASCII character set and replaced with **■**. The FOCAL, 1969 MO{dify} command uses this key to delete all text already output to the system console.

It is apparent this function would have been assigned to an alternate code had the newer displayed character been the usual graphic at the time; most newer software considers this code equivalent to the backspace or delete key as implemented on newer keyboards.

within the ED{it} command, **Control-K** will delete all characters already processed and output to the system console terminal.

Note: If the ED{it} command has not modified the line number, the line number and **<HT>** will be preserved and will output to the system console terminal to continue the editing process. If the line number has already been modified or deleted, it will not be restored after **Control-K** has deleted all processed line contents. If **Control-U** is used to restart the ED{it} command, the line number [and **<HT>**] will be restored as described in 4) above.

- 6) Pressing **Control-H** [**<BS>**] initiates the *rubout* [delete previous character in the line edit buffer] function. This can also be accomplished using the DEL [**<RO>**] key or **■** [originally displayed as **␣** on certain older console terminals].

P?S/8 1976 [Version 8P] commands [continued].

The specific implementation of this function is determined by several keyboard monitor settings. For most hard-copy terminals, each instance echoes the previous character as it is deleted; this basic form of *rubout* handling can be implemented using the VARY RUBOUT ECHO command. If the VARY RUBOUT TOPS10 command is used, all deleted characters will be echoed in reverse order of deletion surrounded by █ characters.

For display terminals, the most widely used method is to output <BS> <SP> <BS> which visually obliterates the deleted character; this can be implemented using the VARY RUBOUT BACKSP command. Additional commands are available for custom support of common DEC terminals such as the VT05, VT52 and VT100. For more information use the VARY /A command for a complete list of terminal customizations and related issues.

EN{ter}:u *filename*:u

A specified [or default from a prior file name command] file name is entered into the directory; no actual file storage is modified.

Appendix A contains information on recovering from accidental file deletion using the EN{ter} and other keyboard monitor commands along with specific system program utilities [if available].

Judicious use of the EN{ter} command can place files for maximum efficiency. On certain tape systems, some programs can access the contents of files faster if the files are arranged in an interleaved manner; the EN{ter} command can setup the proper file placement prior to use.

For example, on TC01/TC08/TC55/TU56 systems, it is impossible to access adjacent files without causing a tape turnaround.

Note: This is a hardware restriction. If a DECTape handler allowed tape motion when the handler exits, the tape would always eventually wind into the forward end zone; the next tape operation would first have to rewind the tape to just below the desired initial tape block, then turn around to cause forward tape motion.

P?S/8 1976 [Version 8P] commands [continued].

Since every call to the system device handler stops the tape upon exit, the next call must be to an area slightly further up the tape yet not immediately adjacent for a forward initial search to be effective. In the specific case of TFS files, this means attempts to access the immediately adjacent file will cause a tape turnaround; accessing a file an additional file slot [or more] further up the tape will start with forward motion and the tape operation will successfully finish without a tape turnaround.

Thus, by arranging the files of a project in an interleaved order, access to the files is greatly increased. The interleave can be accomplished either by having files from an unrelated project interdigitated with the current project files, or the project files can be split into halves, with the second half files between the first half files; there would be no turnarounds in the former case and a single turnaround in the latter case due to having to rewind to access the second half files.

The *slurp* binary loader for TC01/TC08/TU55/TU56 always starts with an initial forward search to more quickly access the binary files, which are generally located further up the tape from the loader code image. The interleaved file approach will decrease file access time for binary files as this further improves the loading process; as such, it would be prudent to interleave binary output files with the source code files of the same project. Both sets of files will be accessed in an optimal way by the programs generally associated with these files [respectively PAL for source code files and the BIN binary file loader for *slurp* binary files]. See Note 3.

HA{lt}

The computer deliberately stops running. Press **Continue** [or equivalent if available] to restart the machine; the keyboard monitor continues in the same state it was prior to the machine halting.

Do not use this command on systems that do not support the **Continue** functionality; the only remedy would be to perform a system restart which would reload the keyboard monitor without saving the edit buffer or other current particulars.

Note: It is merely a vicious and unfounded rumor that the command also causes the machine to catch fire. The PDP-8 supports the HLT instruction to stop program execution; it is the stuff of legend there is the HCF instruction to both Halt and Catch Fire; P?S/8 stops the processor using the HLT instruction which would tend to cool the computer.

P?S/8 1976 [version 8P] commands [continued].

HR{eader} [*optional-starting-line-number*] [*optional-line-increment*]

Auto-sequenced input is initiated starting at the specified *optional-starting-line-number* [line 100 if none given]. Subsequent lines are incremented by the *optional-line-increment* value [default is 10 if none given]. If available, the input characters are read from the high-speed paper-tape reader; otherwise, this command is identical to the SE{quence} command.

Auto-sequence mode can be terminated by **Control-D**. The standard P?S/8 text character set applies; lower-case characters will be "folded" to the upper-case equivalent. See Note 4.

LA{st}

This command outputs the line number of the highest existing line [if any] to the system console terminal. All line numbers must be in the range of 0 through 4094 decimal [0 through 7776 octal].

Note: Perverse resequencing operations can cause an overflow condition. If required, the situation can always be rectified without file loss by using smaller line number increments in a subsequent resequence operation [such as merely using the default values]. If the user has some familiarity with the contents of a particular file, the LA{st} command can confirm the highest line number is in range; if the line numbers overflow, listing the file contents will show lack of proper line number order. After a valid resequencing operation, the file will again show ordered line numbers; the LA{st} command will output the highest line number as expected.

Note: In all releases of P?S/8, resequence operations perform an automatic LA{st} command; if the user is familiar with the file contents, the effects of line number overflow will be immediately evident.

LE{ft}

The free word count within the edit buffer is shown on the system console terminal. See Note 5.

As more text lines are added to the edit buffer, the free word count decreases. A full file condition occurs when there is insufficient free space to insert the latest line contained in the command-line buffer; the **FILE FULL** error message is shown on the system console terminal and the latest command-line contents are lost.

P?S/8 1976 [version 8P] commands [continued].

The LE{ft} command can serve as a warning mechanism indicating how much free space is currently available. If a file is nearly full, corrective remedies can be applied [such as splitting the file into two independent files as detailed in the description of the SC{ratch} command].

All TFS files are 2048 12-bit words [4096 six-bit bytes] long regardless of internal content. Within TFS text files, two 12-bit words are used for line number and line pointer purposes for each file line. All lines start on 12-bit word boundaries. Since each line has its own line pointer, all file modifications are performed nearly instantaneously.

while the line pointers and associated line numbers of all file lines are 12-bit words, the rest of the file predominantly consists of six-bit bytes; lines that end with an odd byte count include a filler byte to maintain 12-bit word alignment. All file lines are contiguously stored with the line pointer and line number values stored at the back end of the file; there is a single 12-bit word set to zero after the last file line to serve as end of file for all programs that only read text characters.

There is a four 12-bit word reserved area; as such, the LE{ft} command will report 2044 words available in an empty file.

A good rule of thumb is to double the free word count to gauge how many additional bytes can be added to the file. It is advisable to have a reasonable reserve within every file for future editing purposes [thus avoiding loss of the command line when attempting to add another line to the file]. Most users should consider SA{ving} the edit buffer to a file when the available space falls below 100 words; the edit buffer can then be cleared with the SC{ratch} command to start the next file. For a typical project, the contents of multiple files passed to any system program are concatenated together as a continuous input stream; a list of up to 17 input files may be passed to any system program.

LI{st} [*line-number-range*]

All file lines in the edit buffer in the specified *line-number-range* are output to the system console terminal.

Note: Output formatting is affected by the current horizontal tab setting as each line of output starts with the line number followed by <HT>. This equally applies to additional <HT> characters within the file.

P?S/8 1976 [version 8P] commands [continued].

LO{ad}:*u filename:u*

A specified [or default from a prior file name command] file name is located within the TFS directory; the file contents are loaded into the edit buffer. Keyboard monitor/integrated line editor commands may be used to examine or modify the contents of the edit buffer. The original file is unchanged unless an explicit SA{ave} command is performed. See Note 6.

Note: Attempts to load nonexistent files will cause error messages.

MO{nitor}

Invoking the MO{nitor} command exits the keyboard monitor; the system is rebooted by restarting the computer at 07600; this is entirely equivalent to a manual restart at the same address.

Note: Using the MO{nitor} command in BATCH files may change the overall flow of BATCH operations depending on specific BATCH options chosen. A BATCH file under development may include the MO{nitor} command to temporarily limit the overall scope of the BATCH file as necessary. Additional information about the P?S/8 BATCH utility is available in other documents.

NU{mber} [*optional-starting-line-number*] [*optional-line-increment*]

Auto-sequenced input is initiated starting at the specified *optional-starting-line-number* [line 100 if none given]. Subsequent lines are incremented by the *optional-line-increment* value [default is 10 if none given]. See Note 7.

During text entry, it is possible to delete all or a portion of the sequence number to execute another command [or perhaps change the line number that applies]. Auto-sequence mode can be terminated by pressing **Control-D**. The standard P?S/8 text character set applies; lower-case characters will be "folded" to the upper-case equivalent.

P?S/8 1976 [version 8P] commands [continued].

PL{ease} [character string]

Executing the PL{ease} command will display the entire command line [including any character string past the command] on the system console terminal. All operations are paused until any non-control character is pressed [which will otherwise be ignored].

A typical usage of the PL{ease} command is within a BATCH file to allow some external action to be applied:

```
.^^ GET PROGRAM LISTING BATCH CONTROL FILE.  
^^  
^^ LAST EDIT: 07-APR-2015 CJL  
^^  
.PLEASE TURN ON THE LINE PRINTER OR PRESS CONTROL-C TO ABORT ...  
[A character is pressed by the operator which is not echoed.]  
^^  
^^ THANK YOU FOR ENABLING THE LINE PRINTER; CONTINUING ...
```

Certain P?S/8 programs output to the system line printer [if available] otherwise falling back on the system console terminal. [Some programs have command-line options to prevent discovery of the line printer which is the default action.] By inserting appropriate lines into the BATCH file similar to the above example, the BATCH can be run as intended.

Note: All of the active control characters are monitored for their normal function during the command pause initiated by the PL{ease} command; the system remains paused until another non-control character is pressed.

Tip: The PL{ease} command can cause the system to be paused while character echo is off. When designing BATCH scripts, it is recommended the control file output appropriate messages before and after executing the PL{ease} command to prevent confusion.

PU{nch} [*line-number-range*]

This command is similar to the LI{st} command except the output is directed to the high-speed paper-tape punch instead of the console terminal; the line numbers [and associated <HT> characters] are suppressed. Punched output consists of 128 standard leader frames [high-bit only set] followed by the contents of the file lines in the specified *line-number-range* then 128 additional standard trailer frames. See Note 4.

Note: In some P?S/8 releases, if the high-speed paper-tape punch is not available, the output is directed to the system console terminal.

P?S/8 1976 [version 8P] commands [continued].

R *system-program:u* [options]

Load the specified *system-program* initial segment into memory and start at the starting address indicated in the system programs directory. Various optional parameters are allowed:

1) Files.

An optional list of output files or input files [or both] can be passed to a system program. Output files [if any] must precede input files [if any]. The maximum number of files is 17. Output files are indicated by `<` after the last output file specification; input files [if any] follow output files [if present].

Note: Specific system programs may further restrict the maximum passed file count.

2) `=xxxx` parameter.

This is a numerical value generally expressed in octal in the range of 0 through 7776 [0 through 4094 decimal]. While 7777 [4095 decimal] cannot be expressed directly, this value is used by default when the `=xxxx` is not used.

Note: Certain system programs allow the use of the current radix in effect for the `=xxxx` parameter; most programs force octal radix.

3) Option switches.

The option switches include `/A` through `/Z` and `/0` through `/9`. When expressing switches in groups, the individual `/` is replaced with a leading `[` followed by one or more option switches, each expressed as a single character, followed by an optional trailing `]`.

All options can be included at any point after the system program name; all values [other than file name lists] can be specified redundantly without error. `<SP>` and `<HT>` characters are freely allowed between values to clarify the command formation if desired.

To maintain backward compatibility with the R-L Monitor System, `,` is allowed as an alternate file name separator [with or without additional `<SP>` or `<HT>` characters]. Input file lists may be preceded with `<` even if no output files are specified.

The position-optimized files `%` and `$` are freely allowed as input [or output] files. New output files will be automatically entered into the TFS directory.

P?S/8 1976 [version 8P] commands [continued].

Logical unit considerations.

The default logical unit for file names is the current logical unit in effect. This may have been changed from the bootup logical unit by commands issued prior to the R command.

The default logical unit for the *system-program* is the bootup logical unit.

Both *system-program* and file specifications can override the prevailing default logical units using explicit *:u* constructs as is the case in various other keyboard monitor commands. Explicit logical unit changes in file specifications update the default value for any additional file specifications on the command line.

By design, all output files are processed before all input files.

Concise program execution.

When the first two characters of the *system-program* name are not in conflict with keyboard monitor commands, the *system-program* may be invoked directly [including all options]. Depending on the P?S/8 system release, this may cause the command process to commence sooner [due to lower internal overhead]; in certain instances, concise program execution may be the only constructive way to use the program due to dependence on the availability of the edit buffer [which is not preserved when the *system-program* is invoked by the R command].

Extended-length file issues.

Most P?S/8 system programs are capable of reading the textual contents of extended-length files created by conversion utilities designed for the specific intersystem process. In the case of the BIGCON utility program, OS/8-format data is read from one DECTape while an extended-length file is created on another.

When P?S/8 Version 8P was released circa 1976, the directory structure for extended-length files was not yet fully defined. As such, extended-length file creation is a very crude and error-prone process.

Instead of a properly-managed block defined in an extended-length files directory, the user designates a standard TFS file's first block as the starting point for writing out extended-length file data. As such, all blocks present on the system device past this file are in danger of being overwritten; the file chosen must be the highest one present in the TFS directory for the conversion to be successful. See Note 8.

P?S/8 1976 [Version 8P] commands [continued].

The upper limit block for writing out the extended-length file is determined by reading the front panel switch register. Since the only known surviving Version 8P system is for TC01/TC08/TU55/TU56 DECTape, a standard DECTape will be used in the example below regarding using the *BIGCON* utility:

Since the highest block on a standard DECTape is 2701, the switch register should be set to 2677, the highest block for a tape size of 2700 blocks. [Due to internal buffering issues within the *BIGCON* utility, the upper limit block should be for a device size that is a multiple of eight blocks.]

Assuming the first file present in the TFS directory is the only one in the directory, the starting block should be 0500 or thereabouts. The following P?S/8 command session illustrates a successful conversion.

Extended-length output file conversion example.

Console output

```
.CA
LGFILE
.ODT LGFILE
7757/0500 E
.R BIGCON,LGFILE</1=7
.
```

Comments

```
CA[ta]log} command.
Contents of TFS directory.
Highest file is LGFILE.
Optional ODT check to
confirm file block.
File block is 0500; exit.
OS/8 data on drive 1.
Conversion complete.
```

The LGFILE now extends from block 0500 through some undetermined block less than 2700.

This file can be used as valid input to programs such as the PAL assembler, which ignores file line numbers [if present]. The entire text of the converted OS/8 file can be referenced by the single file name LGFILE.

The 1976 release of P?S/8 includes the *CONVRT* program to convert the contents of TFS files to/from OS/8 data. The *BIGCON* program can also be used to convert the contents of P?S/8 files to OS/8 data; the OS/8 data can be converted to extended-length files as described above.

Later releases of P?S/8 replaced both programs with the *OS8CON* utility that can convert all the data file variants mentioned; command-line options switches determine the type and direction of the conversion.

RA{dix} [8 or 10]

When the RA{dix} command is invoked without an optional parameter, the radix for all numerical input or output is reversed to the opposite of the current value in effect. This applies to all contexts where a numerical value may be expressed including file line numbers, passing parameters to programs using the =xxxx construct [under certain circumstances] and the numerical output of commands such as LA{st} and LE{ft}.

The radix can be set by passing an explicit parameter: RA{dix} 10 will set the radix to decimal; RA{dix} 8 will set the radix to octal.

Note: The current radix does not affect the interpretation of the passed parameter; the current radix is saved and then set internally to decimal. If the command fails due to improper input, the previous radix is restored.

Changes made with the RA[dix] command revert to the system default value upon system restart; the scope of the RA{dix} command is within the current keyboard monitor session only. While the usual overall default is decimal, this may be changed using the VARY RADIX EIGHT command.

The radix chosen for most work is a user preference setting. Certain limited examples of octal output may be useful to explain basic PDP-8 assembly concepts as illustrated below.

.RADIX 8

.SEQUENCE 177 1

177		*200		/GET THE DESIRED ORIGIN.
200	BEGIN,	TAD	(7)	/GET A <BEL> CHARACTER.
201		TLS		/RING IT.
202		JMP I	(7600)	/RESTART THE SYSTEM NOW.

Note: The above example is not an actual assembly but it does cosmetically resemble a simplified assembly listing. Beginners can learn certain practical aspects of assembly language before mastering additional concepts needed to carry out assembly language programming on a more complete basis. Since the line numbers are permanent [assuming the edit buffer is SA{ved} to a file], small changes can be made which will reflect the way the program actually loads into memory.

P?S/8 1976 [version 8P] commands [continued].

RE{sequence} [*optional-starting-line-number*] [*optional-line-increment*]

The RE[sequence] command reorders the line numbers within the edit buffer as specified by the *optional-starting-line-number* and *optional-line-increment* accordingly. After the resequencing operation [which is carried out virtually instantly], all file lines will be in equal increment order. Note: No changes are made to the text contents of nor the relative order of the lines; however, all future references to the lines must be made with the new line number values.

Resequencing is required when a section of a file has been heavily modified adding in lines with numeric values in between original lines. This can leave little room for additional lines within the section. By setting relatively large line increments, it is possible to set aside sufficient numerical space in the file for the purpose of adding lines [typically with an increment of 1]. Depending on the scope of the editing operation, it may be necessary to resequence the file more than once.

Tip: Assume a file is sequenced with an increment of 10. There is the possibility of adding in lines 101 through 109 between line 100 and line 110. The ED[it] command can be used to create a line 129 from the contents of line 120 [which can then be deleted]. By changing one line, there is now the possibility of adding in lines 101 through 128. If the file is resequenced [or was created with even larger increments] then more than enough line number space can be available for any reasonable edit session.

Note: In certain editing situations, the limiting factor may be file capacity; it may be necessary to first split the file into two independent files before applying localized changes. For more information regarding file splitting, see the description of the SC{ratch} command.

Once a file is stable, it is always recommended to perform a final resequence operation to the default values.

After the resequencing, an automatic LA[st] command is performed. All line numbers must be in the range of 0 through 4094 [0 through 7776 in octal radix].

Note: It is possible to overflow the line number range by either using a starting line number that is too high or a line increment that is too large; this can always be remedied by using smaller parameters such as the default values of starting at line 100 with a line increment of 10.

P?S/8 1976 [Version 8P] commands [continued].

RU{n} *system-program:u* [options]

The RU{n} command is equivalent to the R command; all parameter considerations are identical in both commands. See Note 8.

SA{ve}:*u filename:u*

The edit buffer contents will be saved in the stated [or implied from a prior file name usage] file slot. The TFS directory will be updated first to allocate a new file slot if necessary; if the slot exists, the file will be written out directly. See Note 9.

Note: On systems based on DEctape or other serial access devices, the time savings when updating a file can be profound.

SC{ratch} [*line-number-range*]

All lines in the specified *line-number-range* are removed from the edit buffer. See Note 10.

The SC{ratch} command is often used to split a file into two smaller files when the file is nearly full. A split-point line number is determined to separate the contents of the current file.

Note: Novice users are advised to make a backup copy of the file before attempting this to avoid data loss.

- 1) All lines from the split-point onward can be deleted by using the command SC{ratch} XXXX- [where XXXX is the chosen split-point line number] which creates the smaller file which consists of all lines up to the split-point. The edit buffer should be SA{ved} into the file designated to contain the first part of the former file.
- 2) The complete [former] file should be reloaded using the LO{ad} command; all lines from the beginning of the file through the split-point should be deleted using the command SC{ratch} -YYYY [where YYYY is the value XXXX above less 1] which creates the smaller file which consists of all lines beyond the split-point. The edit buffer should be SA{ved} into the file designated to contain the second part of the former file.

Tip: Since the specific line number used in the ranges given to the SC{ratch} command need not exist [and merely serves as a limit], a line that is exactly one lower than the intended split-point [and non-existent] can be used in both operations when splitting the original file.

P?S/8 1976 [Version 8P] commands [continued].

Tip: The method illustrated is an example of one way to accomplish the file-splitting operation. By using equivalent alternative steps, the same net result can be achieved by first saving the ending portion of the original file then creating the beginning portion.

Tip: Seasoned users will reuse the original file name as one of the two descendant files; only one additional file need be specified within the overall stream of project files; the commands used to pass a stream of files to a system program [PAL, FOCAL, etc.] should be updated accordingly.

Observation: Many competing systems require full-file editing; this means that sufficient space on the storage device is tied up for roughly twice the size of the entire project at the point of any editing operation; later the obsolete copy is released, generally amounting to an inadequate smaller portion of the storage device that eventually leads to serious fragmentation and overhead issues. By limiting editing to fixed size portions of the project, it is only necessary to periodically allocate an additional file at a time to resolve line number problems by splitting the file as described above.

Note: After repeated editing operations on a source file project, certain files will tend to be nearly full while others may be more sparsely populated. This can be remedied by periodically using the CONVRT program to create a single file in OS/8 format, then use CONVRT to recreate the source files from the OS/8 format data; CONVRT standardizes the usage of each file thus eliminating problems of variation in file usage. A report is given of how many files were created; it is conceivable the repacked file set could occupy less files total; however, in rare instances it is conceivable that additional files might be needed due to the standardization. Additional files can be added to the list of output files as necessary. The CONVRT program will indicate how many files were actually used; the rest can be discarded.

SE{sequence} [*optional-starting-line-number*] [*optional-line-increment*]

Auto-sequenced input is initiated starting at the specified *optional-starting-line-number* [line 100 if none given]. Subsequent lines are incremented by the *optional-line-increment* value [default is 10 if none given].

During text entry, it is possible to delete all or a portion of the sequence number to execute another command [or perhaps change the line number that applies]. Auto-sequence mode can be terminated by pressing **Control-D**. The standard P?S/8 text character set applies; lower-case characters will be "folded" to the upper-case equivalent. See Note 11.

SY{stem} :u

The device logical unit **:u** specified in the SY{stem} command is selected as the new logical unit for all subsequent file-oriented logical unit purposes. If the **:u** is omitted or the previous logical unit and the new logical unit are identical, the command is effectively ignored. As required, the corresponding TFS directory on the new logical unit is read into memory replacing the previous one.

Note: For all operations of the keyboard monitor and integrated editor, the TFS directory on the current logical unit is 100% resident in memory; in many instances, this causes an extraordinary decrease in file access time.

The SY{stem} command merely changes the logical unit without otherwise performing another action. See Note 12.

Tip: If a series of operations must be performed on a particular logical unit as specified within a BATCH file, the first operation must effect a change to the intended logical unit. When a BATCH file is created, it is often not possible to determine the intended logical unit until the actual configuration is finalized. The SY{stem} command can be used to initialize the set of commands placed further into the BATCH file stream [all without local logical unit settings] along with other appropriate initialization such as a series of console messages or PL{ease} commands as required. The SY{stem} command can be located in a separate initialization file to modularize the components of the BATCH operation; as many as 17 control files can be passed at one time to BATCH.

TA{b} [0 3 5]

The printing disposition of the **<HT>** character within the keyboard monitor can be customized in various ways to achieve the usual DEC-style alignment to the nearest eight spaces. All **<HT>** characters are affected such as the printed separation between the line number and the file line text, or any **<HT>** characters within the text.

The **<HT>** character can be displayed in one of four operating modes:

- 1) **TA** The **<HT>** is printed as an actual **<HT>** character; the system console terminal must support DEC-style tab stops every eight columns. This is the most efficient operating mode assuming there is hardware support for the **<HT>** character; this is generally true on most modern terminals or terminal emulators.

- 2) **TA 5** This mode is similar to 1) above except **<NUL>** characters are used as filler characters as required for custom support of the Teletype Model 35. The exact number of filler characters required is dependent on the relative position of the carriage within the latest eight-position tab stop. For certain positions, two filler characters are required; for some other positions one filler is sufficient. For two specific positions, no filler is required.

Note: This is the proper method to handle this terminal device; in certain software systems, the Teletype Model 35 is incorrectly handled by always using a single filler character. This is a needless print delay in some instances and inadequate wait time in other situations which will cause printhead ink smearing.

- 3) **TA 3** This mode simulates horizontal tab stops by outputting the proper number of **<SP>** characters to maintain alignment; this is required on printing terminals such as the Teletype Model 33 with no actual horizontal tab support.

This is the slowest operating mode with the single advantage of universally functioning on all console devices.

- 4) **TA 0** The **<HT>** character is printed as one space character; this is an option for very slow hard-copy terminals where the trade-off of print time versus cosmetics may be a factor.

The **TA{b}** command allows temporary override of the permanent setting for **<HT>** printing. Users of slow terminals may opt to change the operating mode from **TA 3** to **TA 0** to more quickly obtain a minimal listing.

The system default setting is **TA{b} 3** for maximum compatibility with most terminals. This can be modified using an appropriate sub-option of the system VARY utility TTY option.

TY{pe} [line-number-range]

All file lines in the edit buffer in the specified *line-number-range* are output to the system console terminal.

The **TY{pe}** command is equivalent to the **LI{ST}** command; all parameter considerations are identical in both commands. See Note 13.

P?S/8 1976 [Version 8P] commands [continued].

^^ [optional comment string]

^^ is a non-action command to the keyboard monitor [and is equivalent to the REM statement as used in most BASIC implementations or C{omment} as implemented in FOCAL, 1969]. ^^ can be used alone or with an optional trailing string of additional characters which are ignored. This is often used as internal documentation within BATCH files.

Tip: The output of BATCH files can be controlled by judicious use of the EC{ho} command. As such, ^^ statements can be used to document internal operations of the BATCH file [with no intention of display during normal operation of the program], or may be deliberately sent to the system console printer as part of the overall BATCH operation. ^^ can also be used in conjunction with PL{ease} commands to acknowledge the operator's response.

P?S/8 1976 [Version 8P] Footnotes.

- 1) The DE{lete} command was replaced with the KI{ll} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310.
- 2) In the 1976 release of P?S/8 [Version 8P], the ED{it} command does not support a control character to process and output all remaining characters in the line. The workaround for this is to use the **Control-A** character as many times as necessary to output the entire line contents. [Note: If no search operation has been performed, a single **Control-A** character will output the entire line. If a search has been performed, multiple usage of the **Control-A** character may be necessary to output the entire line; additional **Control-A** characters are innocuous.] Newer releases of P?S/8 support the **Control-F** character to process and output all remaining characters of the line.
- 3) Binary loading of P?S/8 *slurp* binary files is accomplished using the P?S/8 BIN system program utility. This program actually has several operating modes initiated by various command-line option switches which enable one of several paper-tape utility modes; the default mode is to load the *slurp* loader which is the only exception to the normal permanent residency of the system device handler [other than when loading the binary paper-tape **BIN** and **RIM** loader programs]. The system can still be restarted in the normal manner because a device-specific bootstrap is placed at 07600 when the *slurp* loader is moved over the system device handler. Various loading options are available to enhance the loading process by the use of specific command-line option switches, some of which have equivalent meaning when used in certain of the paper-tape utility modes. Additional information about other functions and options of the BIN program is available in other documents.
- 4) The HR{eader} and PU{nch} commands were removed in subsequent releases of P?S/8. A paper-tape-oriented utility to replace both commands has been planned; this will allow more flexible handling of paper-tape text of any size. A tentative design for the utility is described in Appendix D.
- 5) The value printed out by the LE{ft} command in the 1976 release of P?S/8 is the number of free 12-bit words in the edit buffer. Since this can be somewhat confusing, the reported value was changed to the free byte [six-bit character] count in subsequent releases.
- 6) The LO{ad} command was replaced with the FE{tch} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310.

P?S/8 1976 [Version 8P] Footnotes [continued].

- 7) The NU{mber} command was briefly available in the 1976 release of P?S/8. Although the goal was to transition to greater compatibility with the DIBOL-8 and COS-300/310 systems [while retaining the equivalent SE{quence} command for backward compatibility], the command was removed in subsequent releases of P?S/8 when later documentation revealed the actual COS-300/310 command for auto-sequenced text entry is the LN{umber} command.
- 8) The internal design of the R and RU{n} commands was modified in newer releases of P?S/8 to reduce overhead associated with certain system programs. At issue was the specifics of the original implementation of extended-length file support which is being abandoned in favor of the P?S/8 SHELL overlay. Future releases of P?S/8 will further lower the overhead once support for the simplified extended-length file structure is fully implemented.
- 9) The SA{ve} command was replaced with the WR{ite} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310.
- 10) The SC{ratch} command was replaced with the ER{ase} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310.
- 11) The SE{quence} command was replaced with the LN{umber} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310; the control character to end auto-sequence mode was changed from **Control-D** to **Control-Z** for increased compatibility with a multitude of DEC operating systems.
- 12) The SY{stem} command was replaced with the LC{atalog} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310.
- 13) The TY{pe} command was removed in favor of the LI{st} command in subsequent releases of P?S/8 for increased compatibility with DIBOL-8 and COS-300/310. Users of the 1976 release of P?S/8 are encouraged to use the LI{st} command in anticipation of upgrading to a newer release.

P?S/8 1989 [Version 8Z] commands.

CA{talog} :*u*

Obtain a quick TFS directory listing on the system console printer.

Note: No storage device access is required unless the optional :*u* is used; this forces the TFS directory to be read into memory from the stated logical device unit; the TFS directory is entirely cached during all keyboard monitor operations.

The output format is four file names per line. Since the maximum number of files allowed is 84, the largest possible printout is 21 lines.

Note: Allocation of storage for files and system programs varies. The P?S/8 Version 8Z release for TC01/TC08/TU55/TU56, as used with standard-length DECTapes, allows up to 60 files. Longer DECTapes will allow additional files up to the maximum of 84 files.

Files are displayed in physical order of storage on the current logical device unit. Empty entries show as <FREE> and will always be indicated unless physically last in the directory [and thus the first of an indefinite number of inactive directory slots not shown].

Note: Multiple EN{ter} and KI{ll} command operations can lead to displaying a series of trailing empty entries in unusual circumstances. The CA{talog} command will print out deleted file entries as empty entries; only the very last entry [or the first entry never used] is completely empty in the sense of either having never been used, or cleared as a special case handled by the KI{ll} command. Trailing completely empty entries are not displayed. For more information regarding empty directory entries, see the description of the KI{ll} command.

EC{ho} [optional 0 or 1]

Inverts the state of output of command characters if no argument given. EC{ho} 0 forces all output off; EC{ho} 1 forces all output on. Pressing **Control-O** during any output operation in progress overrides this by immediately reversing the echo state for each **Control-O** pressed. This command is generally used within BATCH control files to limit output to the system console.

ED{it} [pre-existing line number]

ED{it} the contents of a specified line in the edit buffer. The ED{it} command is operated through the use of certain control characters unique to the ED{it} command [as well as most of the control characters that are used throughout the command processor].

The operation of the ED{it} command is patterned after the FOCAL, 1969 MO{dify} command with accommodations to the P?S/8 editing environment; this includes features that apply to several operating modes [including the automatic placement of line numbers while entering text]. All control functions operate consistently regardless of mode; additional control characters also apply exclusively to the ED{it} command. Description of the applicable control characters follows:

- 1) within the P?S/8 ED{it} command and the FOCAL, 1969 MO{dify} command, pressing **Control-G** initiates the search mode; the next character typed is searched for from the current cursor position until either a matching character is found or the end of the line is reached.

In FOCAL, 1969, the MO{dify} operation starts as if **Control-G** were already pressed; the P?S/8 ED{it} command starts in character insertion mode after the line number and a **<HT>** character are printed on the system console.

Note: For more information on the exact printing of the **<HT>** character, see the description of the TA{b} command.

At any point during the ED{it} command operation, all keyboard monitor functions are allowed including the insertion and deletion of text characters. This can effect direct editing of the line number characters [and trailing **<HT>** separator character] if desired. It is possible to create alternate line numbers with contents based on the specified line number without changing the contents of the specified line. With the line number deleted, it is possible to create keyboard monitor commands to be executed in place based on the contents of the original line; this facilitates using complex commands saved within text files avoiding the need for manual entry for each usage.

P?S/8 1989 [version 8Z] commands [continued].

Note: It is also possible to create files containing a single complex command; these files can be used with P?S/8 BATCH without any form of editing. By editing out all other lines in a file containing multiple complex commands, a temporary file can be created containing only the desired line; depending on the nature of the command, it may be appropriate to save the file as one of the positional files \$ or % due to their shorter access time.

In FOCAL, 1969, the MO{dify} command will completely process and close out the specified line if the search does not match; when this is unexpected, it can be quite disruptive to the editing process. Additional MO{dify} or other commands will be needed to resolve unforeseen issues.

within the P?S/8 ED{it} command, a failing search merely processes and outputs all the rest of the characters in the specified line [if any] and waits for further input [which could include various abort or restart operations thus preserving the original contents of the specified line].

- 2) In FOCAL, 1969, **Control-L** is used to search for the same character as used in previous searches. In the ED{it} command, **Control-A** is used to search for the same character as used in previous searches. Note: If no prior searches have been performed, **Control-A** will output the rest of the line contents [if any] to the system console terminal and wait for further input.
- 3) In FOCAL, 1969, **Control-J** [**<LF>**] processes and prints out all remaining characters in the line on the system console terminal; additional editing can be performed ending with **Control-M** [**<CR>**] to close out the line. In recent releases of P?S/8, the ED{it} command supports **Control-F** to perform the identical function.
- 4) within the ED{it} command, **Control-J** [**<LF>**] will output **<CR>** and **<LF>** and then the current tentative line contents to the system console terminal. This is especially useful throughout the keyboard monitor when using hard-copy console terminals which generally require the display of deleted characters [which can be confusing].
- 5) **Control-U** will completely restart the editing operation; this includes the printing of the opening line number and **<HT>** characters on the system console terminal. No changes have been made to the specified line.

- 6) When FOCAL, 1969 was designed, the displayed character associated with ASCII code 95 decimal [137 octal] [5F hex] was implemented on early Teletype Model 33 and similar console terminals as ; the displayed character was later phased out of the ASCII character set and replaced with . The FOCAL, 1969 MO{dify} command uses this key to delete all text already output to the system console.

It is apparent this function would have been assigned to an alternate code had the newer displayed character been the usual graphic at the time; most newer software considers this code equivalent to the backspace or delete key as implemented on newer keyboards.

Within the ED{it} command, **Control-K** will delete all characters already processed and output to the system console terminal.

Note: If the ED{it} command has not modified the line number, the line number and **<HT>** will be preserved and will output to the system console terminal to continue the editing process. If the line number has already been modified or deleted, it will not be restored after **Control-K** has deleted all processed line contents. If **Control-U** is used to restart the ED{it} command, the line number [and **<HT>**] will be restored as described in 5) above.

- 7) Pressing **Control-H** [**<BS>**] initiates the *rubout* [delete previous character in the line edit buffer] function. This can also be accomplished using the DEL [**<RO>**] key or  [originally displayed as  on certain older console terminals].

The specific implementation of this function is determined by several keyboard monitor settings. For most hard-copy terminals, each instance echoes the previous character as it is deleted; this basic form of *rubout* handling can be implemented using the SET RUBOUT ECHO command. If the SET RUBOUT TOPS10 command is used, all deleted characters will be echoed in reverse order of deletion surrounded by  characters.

For display terminals, the most widely used method is to output **<BS>** **<SP>** **<BS>** which visually obliterates the deleted character; this can be implemented using the SET RUBOUT BACKSP command. Additional commands are available for custom support of common DEC terminals such as the VT05, VT52 and VT100. For more information use the SET /A command for a complete list of terminal customizations and related issues.

P?S/8 1989 [Version 8Z] commands [continued].

EN{ter}:*u filename:u*

A specified [or default from a prior file name command] file name is entered into the directory; no actual file storage is modified.

Appendix A contains information on recovering from accidental file deletion using the EN{ter} and other keyboard monitor commands along with specific system program utilities [if available].

Judicious use of the EN{ter} command can place files for maximum efficiency. On certain tape systems, some programs can access the contents of files faster if the files are arranged in an interleaved manner; the EN{ter} command can setup the proper file placement prior to use.

For example, on TC01/TC08/TC55/TU56 systems, it is impossible to access adjacent files without causing a tape turnaround. [Note: This is a hardware restriction. If a DECTape handler allowed tape motion when the handler exits, the tape would always eventually wind into the forward end zone; the next tape operation would first have to rewind the tape to just below the desired initial tape block, then turn around to cause forward tape motion.

Since every call to the system device handler stops the tape upon exit, the next call must be to an area slightly further up the tape yet not immediately adjacent for a forward initial search to be effective. In the specific case of TFS files, this means attempts to access the immediately adjacent file will cause a tape turnaround; accessing a file an additional file slot [or more] further up the tape will start with forward motion and the tape operation will successfully finish without a tape turnaround.

Thus, by arranging the files of a project in an interleaved order, access to the files is greatly increased. The interleave can be accomplished either by having files from an unrelated project interdigitated with the current project files, or the project files can be split into halves, with the second half files between the first half files; there would be no turnarounds in the former case and a single turnaround in the latter case due to having to rewind to access the second half files.

The *slurp* binary loader for TC01/TC08/TU55/TU56 always starts with an initial forward search to more quickly access the binary files, which are generally located further up the tape from the loader code image. The interleaved file approach will decrease file access time for binary files as this further improves the loading process; as such, it would be prudent to interleave binary output files with the source code files of the same project. Both sets of files will be accessed in an optimal way by the programs generally associated with these files [respectively PAL for source code files and the BIN binary file loader for *slurp* binary files]. See Note 1.

ER{ase} [*line-number-range*]

All lines in the specified *line-number-range* are removed from the edit buffer.

The ER{ase} command is often used to split a file into two smaller files when the file is nearly full. A split-point line number is determined to separate the contents of the current file.

Note: Novice users are advised to make a backup copy of the file before attempting this to avoid data loss.

- 1) All lines from the split-point onward can be deleted by using the command ER{ase} XXXX- [where XXXX is the chosen split-point line number] which creates the smaller file which consists of all lines up to the split-point. The edit buffer should be WR{itten} into the file designated to contain the first part of the former file.
- 2) The complete [former] file should be reloaded using the FE[tch] command; all lines from the beginning of the file through the split-point should be deleted using the command ER{ase} -YYYY [where YYYY is the value XXXX above less 1] which creates the smaller file which consists of all lines beyond the split-point. The edit buffer should be WR{itten} into the file designated to contain the second part of the former file.

Tip: Since the specific line number used in the ranges given to the ER{ase} command need not exist [and merely serves as a limit], a line that is exactly one lower than the intended split-point [and non-existent] can be used in both operations when splitting the original file.

Tip: The method illustrated is an example of one way to accomplish the file-splitting operation. By using equivalent alternative steps, the same net result can be achieved by first saving the ending portion of the original file then creating the beginning portion.

P?S/8 1989 [Version 8Z] commands [continued].

Tip: Seasoned users will reuse the original file name as one of the two descendant files; only one additional file need be specified within the overall stream of project files; the commands used to pass a stream of files to a system program [PAL, FOCAL, etc.] should be updated accordingly.

Observation: Many competing systems require full-file editing; this means that sufficient space on the storage device is tied up for roughly twice the size of the entire project at the point of any editing operation; later the obsolete copy is released, generally amounting to an inadequate smaller portion of the storage device that eventually leads to serious fragmentation and overhead issues. By limiting editing to fixed size portions of the project, it is only necessary to periodically allocate an additional file at a time to resolve line number problems by splitting the file as described above.

Note: After repeated editing operations on a source file project, certain files will tend to be nearly full while others may be more sparsely populated. This can be remedied by periodically using the OS8CON program to create a single file in OS/8 format, then use OS8CON to recreate the source files from the OS/8 format data; OS8CON standardizes the usage of each file thus eliminating problems of variation in file usage. A report is given of how many files were created; it is conceivable the repacked file set could occupy less files total; however, in rare instances it is conceivable that additional files might be needed due to the standardization. Additional files can be added to the list of output files as necessary. The OS8CON program will indicate how many files were actually used; the rest can be discarded.

`FE{tch}:u filename:u`

A specified [or default from a prior file name command] file name is located within the TFS directory; the file contents are loaded into the edit buffer. Keyboard monitor/integrated line editor commands may be used to examine or modify the contents of the edit buffer. The original file is unchanged unless an explicit `WR{ite}` command is performed.

Note: Attempts to fetch nonexistent files will cause error messages.

`HA{!t}`

The computer deliberately stops running. Press `Continue` [or equivalent if available] to restart the machine; the keyboard monitor continues in the same state it was prior to the machine halting.

P?S/8 1989 [Version 8Z] commands [continued].

Do not use this command on systems that do not support the Continue functionality; the only remedy would be to perform a system restart which would reload the keyboard monitor without saving the edit buffer or other current particulars.

Note: It is merely a vicious and unfounded rumor that the command also causes the machine to catch fire. The PDP-8 supports the HLT instruction to stop program execution; it is the stuff of legend there is the HCF instruction to both Halt and Catch Fire; P?S/8 stops the processor using the HLT instruction which would tend to cool the computer.

KI{ll}:*u filename:u*

KI{ll} [delete] a file from the TFS directory. The affected file name is removed from the TFS directory; the underlying file contents are not affected, making it possible to restore the file if the deletion was unintended.

Note: When a file is deleted from the TFS directory, the free directory slot is available for storage of another file; if this should occur, the original file is lost.

Note: The KI{ll} command does not check to potentially clear the previous entry after clearing what becomes the first of any final completely empty entries. This is a deliberate design decision to aid in file recovery; deleted entries generally contain partial file names [unless the file name was only one or two characters]. Appendix A contains information on recovering from accidental file deletion using keyboard monitor commands and specific system program utilities [if available].

LA{st}

This command outputs the line number of the highest existing line [if any] to the system console terminal. All line numbers must be in the range of 0 through 4094 decimal [0 through 7776 octal].

Note: Perverse resequencing operations can cause an overflow condition. If required, the situation can always be rectified without file loss by using smaller line number increments in a subsequent resequence operation [such as merely using the default values]. If the user has some familiarity with the contents of a particular file, the LA{st} command can confirm the highest line number is in range; if the line numbers overflow, listing the file contents will show lack of proper line number order. After a valid resequencing operation, the file will again show ordered line numbers; the LA{st} command will output the highest line number as expected.

Note: In all releases of P?S/8, resequence operations perform an automatic LA{st} command; if the user is familiar with the file contents, the effects of line number overflow will be immediately evident.

LC{atalog} :*u*

The device logical unit :*u* specified in the LC{atalog} command is selected as the new logical unit for all subsequent file-oriented logical unit purposes. If the :*u* is omitted or the previous logical unit and the new logical unit are identical, the command is effectively ignored. As required, the corresponding TFS directory on the new logical unit is read into memory replacing the previous one.

Note: For all operations of the keyboard monitor and integrated editor, the TFS directory on the current logical unit is 100% resident in memory; in many instances, this causes an extraordinary decrease in file access time.

The LC{atalog} command merely changes the logical unit without otherwise performing another action.

Tip: If a series of operations must be performed on a particular logical unit as specified within a BATCH file, the first operation must effect a change to the intended logical unit. When a BATCH file is created, it is often not possible to determine the intended logical unit until the actual configuration is finalized. The LC{atalog} command can be used to initialize the set of commands placed further into the BATCH file stream [all without local logical unit settings] along with other appropriate initialization such as a series of console messages or PL{ease} commands as required. The LC{atalog} command can be located in a separate initialization file to modularize the components of the BATCH operation; as many as 17 control files can be passed at one time to BATCH.

LE{ft}

The free byte [six-bit character] count within the edit buffer is shown on the system console terminal.

As more text lines are added to the edit buffer, the free byte count decreases. A full file condition occurs when there is insufficient free space to insert the latest line contained in the command-line buffer; the **FILE FULL** error message is shown on the system console terminal and the latest command-line contents are lost.

P?S/8 1989 [version 8Z] commands [continued].

The LE{ft} command can serve as a warning mechanism indicating how much free space is currently available. If a file is nearly full, corrective remedies can be applied [such as splitting the file into two independent files as detailed in the description of the ER{ase} command].

All TFS files are 2048 12-bit words [4096 six-bit bytes] long regardless of internal content. Within TFS text files, two 12-bit words are used for line number and line pointer purposes for each file line. All lines start on 12-bit word boundaries. Since each line has its own line pointer, all file modifications are performed nearly instantaneously.

while the line pointers and associated line numbers of all file lines are 12-bit words, the rest of the file predominantly consists of six-bit bytes; lines that end with an odd byte count include a filler byte to maintain 12-bit word alignment. All file lines are contiguously stored with the line pointer and line number values stored at the back end of the file; there is a single 12-bit word set to zero after the last file line to serve as end of file for all programs that only read text characters.

There is a four 12-bit word reserved area; as such, the LE{ft} command will report 4088 bytes available in an empty file.

It is advisable to have a reasonable reserve within every file for future editing purposes [thus avoiding loss of the command line when attempting to add another line to the file]. Most users should consider WR{iting} the edit buffer to a file when the available space falls below 200 bytes; the edit buffer can then be cleared with the ER{ase} command to start the next file. For a typical project, the contents of multiple files passed to any system program are concatenated together as a continuous input stream; a list of up to 17 input files may be passed to any system program.

LI{st} [*line-number-range*]

All file lines in the edit buffer in the specified *line-number-range* are output to the system console terminal.

Note: Output formatting is affected by the current horizontal tab setting as each line of output starts with the line number followed by <HT>. This equally applies to additional <HT> characters within the file.

LN{umber} [*optional-starting-line-number*] [*optional-line-increment*]

Auto-sequenced input is initiated starting at the specified *optional-starting-line-number* [line 100 if none given]. Subsequent lines are incremented by the *optional-line-increment* value [default is 10 if none given].

P?S/8 1989 [version 8Z] commands [continued].

During text entry, it is possible to delete all or a portion of the sequence number to execute another command [or perhaps change the line number that applies]. Auto-sequence mode can be terminated by pressing **Control-D**. The standard P?S/8 text character set applies; lower-case characters will be "folded" to the upper-case equivalent.

LT{ext} [*line-number-range*]

The contents of all lines in the specified *line-number-range* are directed to the system console terminal without the associated line numbers.

This is the fastest form of file contents display, which can be helpful on slow system console terminals such as the Teletype Model 33.

Note: The command name is derived from the action of listing only the text in the file.

Tip: To decrease printing time still further [at the expense of cosmetic formatting], horizontal tab disposition should be set to output a single space character for every **<HT>** character in the file with the TA{b} 0 command [or the analogous SET TAB SINGLE utility program option if it is desirable to permanently change the default setting].

MO{nitor}

Invoking the MO{nitor} command exits the keyboard monitor; the system is rebooted by restarting the computer at 07600; this is entirely equivalent to a manual restart at the same address.

Note: Using the MO{nitor} command in BATCH files may change the overall flow of BATCH operations depending on specific BATCH options chosen. A BATCH file under development may include the MO{nitor} command to temporarily limit the overall scope of the BATCH file as necessary. Additional information about the P?S/8 BATCH utility is available in other documents.

PL{ease} [character string]

Executing the PL{ease} command will display the entire command line [including any character string past the command] on the system console terminal. All operations are paused until any non-control character is pressed [which will otherwise be ignored].

A typical usage of the PL{ease} command is within a BATCH file to allow some external action to be applied:

P?S/8 1989 [version 8Z] commands [continued].

```
.^^ GET PROGRAM LISTING BATCH CONTROL FILE.  
^^  
^^ LAST EDIT: 07-APR-2015 CJL  
^^  
.PLEASE TURN ON THE LINE PRINTER OR PRESS CONTROL-C TO ABORT ...  
[A character is pressed by the operator which is not echoed.]  
^^  
^^ THANK YOU FOR ENABLING THE LINE PRINTER; CONTINUING ...
```

Certain P?S/8 programs output to the system line printer [if available] otherwise falling back on the system console terminal. [Some programs have command-line options to prevent discovery of the line printer which is the default action.] By inserting appropriate lines into the BATCH file similar to the above example, the BATCH can be run as intended.

Note: All of the active control characters are monitored for their normal function during the command pause initiated by the PL{ease} command; the system remains paused until another non-control character is pressed.

Tip: The PL{ease} command can cause the system to be paused while character echo is off. When designing BATCH scripts, it is recommended the control file output appropriate messages before and after executing the PL{ease} command to prevent confusion.

R *system-program:u* [options]

Load the specified *system-program* initial segment into memory and start at the starting address indicated in the system programs directory. Various optional parameters are allowed:

1) Files.

An optional list of output files or input files [or both] can be passed to a system program. Output files [if any] must precede input files [if any]. The maximum number of files is 17. Output files are indicated by  after the last output file specification; input files [if any] follow output files [if present].

Note: Specific system programs may further restrict the maximum passed file count.

2) =xxxx parameter.

This is a numerical value generally expressed in octal in the range of 0 through 7776 [0 through 4094 decimal]. While 7777 [4095 decimal] cannot be expressed directly, this value is used by default when the =xxxx is not used.

P?S/8 1989 [version 8Z] commands [continued].

Note: Certain system programs allow the use of the current radix in effect for the =xxxx parameter; most programs force octal radix.

3) Option switches.

The option switches include /A through /Z and /0 through /9. When expressing switches in groups, the individual / is replaced with a leading [followed by one or more option switches, each expressed as a single character, followed by an optional trailing].

All options can be included at any point after the system program name; all values [other than file name lists] can be specified redundantly without error. <SP> and <HT> characters are freely allowed between values to clarify the command formation if desired.

To maintain backward compatibility with the R-L Monitor System, , is allowed as an alternate file name separator [with or without additional <SP> or <HT> characters]. Input file lists may be preceded with < even if no output files are specified.

The position-optimized files % and \$ are freely allowed as input [or output] files. New output files will be automatically entered into the TFS directory.

Choice of input and output file specification.

As of the release of P?S/8 Version 8Y, the R command can also be expressed with the lists of files reversed. Input files can be specified first followed by > then the output files. Command formation is now more flexible and reminiscent of certain newer operating systems.

Seasoned users tend to prefer the original implementation; newer users will likely prefer specifying input files first since many modern operating systems form analogous commands in this manner.

Note: By having both methods of file specification available, P?S/8 takes a neutral stand on this design issue; however, regardless of command form, output files are always processed before input files as discussed below.

Logical unit considerations.

The default logical unit for file names is the current logical unit in effect. This may have been changed from the bootup logical unit by commands issued prior to the R command.

The default logical unit for the *system-program* is the bootup logical unit.

P?S/8 1989 [Version 8Z] commands [continued].

Both *system-program* and file specifications can override the prevailing default logical units using explicit *:u* constructs as is the case in various other keyboard monitor commands. Explicit logical unit changes in file specifications update the default value for any additional file specifications on the command line.

By design, all output files are processed before all input files.

Concise program execution.

When the first two characters of the *system-program* name are not in conflict with keyboard monitor commands, the *system-program* may be invoked directly [including all options]. Depending on the P?S/8 system release, this may cause the command process to commence sooner [due to lower internal overhead]; in certain instances, concise program execution may be the only constructive way to use the program due to dependence on the availability of the edit buffer [which is not preserved when the *system-program* is invoked by the R command].

Extended-length File Issues.

Most P?S/8 system programs are capable of reading the textual contents of extended-length files created by conversion utilities designed for the specific intersystem process addressed. The OS8CON conversion utility can access data on a logical drive unit of the P?S/8 system device handler if certain compatibility requirements are met [which is often the case]. Other utilities such as L6DCON require internal device handlers due to the inherent incompatibility of the underlying media [256 words/block LINCtapes on either the LINC-8 or the PDP-12]. Appendix D described a tentative program to transmit paper-tape text data to/from P?S/8 including the optional use of extended-length files. In all such programs, extended-length output files can be created specifying particular command-line option switches. After conversion or other processing is completed, extended-length files can be passed as input files to most system programs [such as PAL, DUMP and FOCAL] that ignore file line numbers.

All of these programs properly handle extended-length output files using the definition of the extended directory associated with P?S/8 Version 8Y. Future releases will streamline the implementation to conform with new structures associated with the release of P?S/8 Version 9A; there will be minimal impact on the user once the new structures are fully supported. See Note 2.

Note: There had been plans to allow full support of multiple extended-length files per device; as such, there were partially-implemented structures added as a framework for this effort.

P?S/8 1989 [version 8Z] commands [continued].

This line of system development has been abandoned in favor of the P?S/8 SHELL overlay, which will provide more comprehensive directory support using an entirely different user interface model.

Future P?S/8 releases will support logical device units with a single extended-length file which will be compatible with all basic P?S/8 system programs that ignore file line numbers [such as BATCH, BLKCPY, DUMP, FOCAL, PAL and all prevailing filesystem conversion programs]. The SET utility will be upgraded to provide enhanced support of extended-length files including user file name preference as well as enhanced raw support for use with programs such as the PAL symbolic cross-reference facility.

RA{dix} [8 or 10]

When the RA{dix} command is invoked without an optional parameter, the radix for all numerical input or output is reversed to the opposite of the current value in effect. This applies to all contexts where a numerical value may be expressed including file line numbers, passing parameters to programs using the =xxxx construct [under certain circumstances] and the numerical output of commands such as LA{st} and LE{ft}.

The radix can be set by passing an explicit parameter: RA{dix} 10 will set the radix to decimal; RA{dix} 8 will set the radix to octal.

Note: The current radix does not affect the interpretation of the passed parameter; the current radix is saved and then set internally to decimal. If the command fails due to improper input, the previous radix is restored.

Changes made with the RA[dix] command revert to the system default value upon system restart; the scope of the RA{dix} command is within the current keyboard monitor session only. While the usual overall default is decimal, this may be changed using the SET RADIX EIGHT command.

The radix chosen for most work is a user preference setting. Certain limited examples of octal output may be useful to explain basic PDP-8 assembly concepts as illustrated below.

.RADIX 8

.SEQUENCE 177 1

177		*200		/GET THE DESIRED ORIGIN.
200	BEGIN,	TAD	(7)	/GET A <BEL> CHARACTER.
201		TLS		/RING IT.
202		JMP I	(7600)	/RESTART THE SYSTEM NOW.

Note: The above example is not an actual assembly but it does cosmetically resemble a simplified assembly listing. Beginners can learn certain practical aspects of assembly language before mastering additional concepts needed to carry out assembly language programming on a more complete basis. Since the line numbers are permanent [assuming the edit buffer is WR{itten} to a file], small changes can be made which will reflect the way the program actually loads into memory.

RE{sequence} [*optional-starting-line-number*] [*optional-line-increment*]

The RE[sequence] command reorders the line numbers within the edit buffer as specified by the *optional-starting-line-number* and *optional-line-increment* accordingly. After the resequencing operation [which is carried out virtually instantly], all file lines will be in equal increment order. Note: No changes are made to the text contents of nor the relative order of the lines; however, all future references to the lines must be made with the new line number values.

Resequencing is required when a section of a file has been heavily modified adding in lines with numeric values in between original lines. This can leave little room for additional lines within the section. By setting relatively large line increments, it is possible to set aside sufficient numerical space in the file for the purpose of adding lines [typically with an increment of 1]. Depending on the scope of the editing operation, it may be necessary to resequence the file more than once.

Tip: Assume a file is sequenced with an increment of 10. There is the possibility of adding in lines 101 through 109 between line 100 and line 110. The ED[it] command can be used to create a line 129 from the contents of line 120 [which can then be deleted]. By changing one line, there is now the possibility of adding in lines 101 through 128. If the file is resequenced [or was created with even larger increments] then more than enough line number space can be available for any reasonable edit session.

Note: In certain editing situations, the limiting factor may be file capacity; it may be necessary to first split the file into two independent files before applying localized changes. For more information regarding file splitting, see the description of the ER{ase} command.

Once a file is stable, it is always recommended to perform a final resequence operation to the default values.

After the resequencing, an automatic LA[st] command is performed. All line numbers must be in the range of 0 through 4094 [0 through 7776 in octal radix].

Note: It is possible to overflow the line number range by either using a starting line number that is too high or a line increment that is too large; this can always be remedied by using smaller parameters such as the default values of starting at line 100 with a line increment of 10.

`RU{n} system-program:u [options]`

The `RU{n}` command is equivalent to the `R` command; all parameter considerations are identical in both commands. See Note 2.

`TA{b} [0 3 5]`

The printing disposition of the `<HT>` character within the keyboard monitor can be customized in various ways to achieve the usual DEC-style alignment to the nearest eight spaces. All `<HT>` characters are affected such as the printed separation between the line number and the file line text, or any `<HT>` characters within the text.

The `<HT>` character can be displayed in one of four operating modes:

- 1) `TA` The `<HT>` is printed as an actual `<HT>` character; the system console terminal must support DEC-style tab stops every eight columns. This is the most efficient operating mode assuming there is hardware support for the `<HT>` character; this is generally true on most modern terminals or terminal emulators.
- 2) `TA 5` This mode is similar to 1) above except `<NUL>` characters are used as filler characters as required for custom support of the Teletype Model 35. The exact number of filler characters required is dependent on the relative position of the carriage within the latest eight-position tab stop. For certain positions, two filler characters are required; for some other positions one filler is sufficient. For two specific positions, no filler is required.

Note: This is the proper method to handle this terminal device; in certain software systems, the Teletype Model 35 is incorrectly handled by always using a single filler character. This is a needless print delay in some instances and inadequate wait time in other situations which will cause printhead ink smearing.

- 3) **TA 3** This mode simulates horizontal tab stops by outputting the proper number of **<SP>** characters to maintain alignment; this is required on printing terminals such as the Teletype Model 33 with no actual horizontal tab support.

This is the slowest operating mode with the single advantage of universally functioning on all console devices.

- 4) **TA 0** The **<HT>** character is printed as one space character; this is an option for very slow hard-copy terminals where the trade-off of print time versus cosmetics may be a factor.

The **TA{b}** command allows temporary override of the permanent setting for **<HT>** printing. Users of slow terminals may opt to change the operating mode from **TA 3** to **TA 0** to more quickly obtain a minimal listing.

The system default setting is **TA{b} 3** for maximum compatibility with most terminals. This can be modified using an appropriate sub-option of the system SET utility TAB option.

WR{ite} :u filename :u

The edit buffer contents will be saved in the stated [or implied from a prior file name usage] file slot. The TFS directory will be updated first to allocate a new file slot if necessary; if the slot exists, the file will be written out directly.

Note: On systems based on DECTape or other serial access devices, the time savings when updating a file can be profound.

^^ [optional comment string]

^^ is a non-action command to the keyboard monitor [and is equivalent to the REM statement as used in most BASIC implementations or C{omment} as implemented in FOCAL, 1969]. **^^** can be used alone or with an optional trailing string of additional characters which are ignored. This is often used as internal documentation within BATCH files.

Tip: The output of BATCH files can be controlled by judicious use of the **EC{ho}** command. As such, **^^** statements can be used to document internal operations of the BATCH file [with no intention of display during normal operation of the program], or may be deliberately sent to the system console printer as part of the overall BATCH operation. **^^** can also be used in conjunction with **PL{ease}** commands to acknowledge the operator's response.

- 1) Binary loading of P?S/8 *slurp* binary files is accomplished using the P?S/8 BIN system program utility. This program actually has several operating modes initiated by various command-line option switches which enable one of several paper-tape utility modes; the default mode is to load the *slurp* loader which is the only exception to the normal permanent residency of the system device handler [other than when loading the binary paper-tape **BIN** and **RIM** loader programs]. The system can still be restarted in the normal manner because a device-specific bootstrap is placed at 07600 when the *slurp* loader is moved over the system device handler. Various loading options are available to enhance the loading process by the use of specific command-line option switches, some of which have equivalent meaning when used in certain of the paper-tape utility modes. Additional information about other functions and options of the BIN program is available in other documents.
- 2) In an ongoing development effort to lower the internal overhead of the R and RU{n} operation which was started in earlier releases of P?S/8, future releases will handle extended-length files in a manner consistent with the planned simplified directory structure. Overall command execution and keyboard monitor swaps [where relevant] will occur faster; in certain instances, system programs that act as overlays to the keyboard monitor will have increased access to resident routines. There will be no impact on the user interface once these changes are implemented.

Appendix A.

Recovery from accidental deletion of TFS files.

As is the case in any file system, it is possible to delete TFS files as the result of unintended acts. Various strategies will be offered to maximize the possibility of proper file recovery.

- 1) Recovering a file accidentally deleted while the file is in the edit buffer.

If the file is still available in the edit buffer, it can be saved to the TFS directory by executing the appropriate file save operation [the SA{ve} command in older releases, the WR{ite} command in newer releases].

For most users of P?S/8, this is the only remedial action needed. However, if file placement is an issue, it may be necessary to place dummy file entries to occupy empty file slots before the intended storage blocks. The CA[ta]log command can be used to confirm the existence of empty entries prior to the desired storage blocks. The EN[ter] command can be used as many times as necessary to achieve the desired results. [Note: Precise file placement can be deferred as long as the file is properly saved somewhere within the TFS directory structure.]

- 2) Recovering a file accidentally deleted and the only known copy is stored in a TFS directory slot.

No file recovery is possible if the file slot is overwritten; as long as the file slot is available with the original file contents, recovery is still possible.

If the proper file slot is evident, an EN{ter} command can be issued to recover the file contents and original file name. The file must be brought into the edit buffer for verification of the expected contents.

Note: The EN{ter} command does not check the contents of the corresponding file storage blocks. Under perverse circumstances, the system may produce junk output from the LI{st} or related commands, or possibly hang the keyboard monitor program when the contents of the corresponding blocks are loaded based on presumption of file validity that turn out to be wrong. Uninitialized tape blocks do not represent a proper TFS text file. By design, no portion of the system device will be accidentally written in such a case; it will be necessary to restart the system at 07600 or other alternate startup methods.

Appendix A [Continued].

As of this writing, there are no utilities to reveal the name of partially deleted files; future releases of the P?S/8 DIRECT utility will add in the ability to view the file name remnant. [Note: Deleted TFS files do not retain the first two file name characters; any additional characters in the file name can be viewed with a proper utility. In the near-term, advanced users of the P?S/8 DUMP program can carry out file name repairs using the ZAP function.]

If there is sufficient confusion regarding the proper file slot for recovery, the recommended procedure is to fill all available file slots with dummy names using the EN{ter} command as many times as needed. If available, the P?S/8 DIRECT utility can identify the file type of each file by the use of appropriate command-line options. [Note: This operation can take significant time to execute; every file is loaded and checked for internal signatures.]

If desired, all slots in the TFS directory can be filled with dummy names prior to use. This will allow any relative file placement for optimization purposes such as the method detailed in the description of the EN{ter} command.

With every available file slot filled, file recovery will generally become a trivial task; file deletion will readily reveal empty file slots.

Future releases of the FILMAN utility may support aliased entry points for common file operations such as MOVE and RENAME. An UNDELE{TE} alias can be added that could guide the user through file recovery limiting choices to partially matching deleted file names only. [Note: It will be a challenge to implement certain of these features unless the operational name is changed or existing keyboard monitor commands are renamed; however, the internal commands of FILMAN can be expanded to support such operations within the context of the FILMAN utility itself.]

End of Appendix A.

Appendix B.

P?S/8 keyboard monitor command names and related design considerations.

The P?S/8 keyboard monitor and integrated editor commands are stored internally in block 0013 of the system device as a table of two word entries:

- 1) A 12-bit word containing the six-bit code for the command [inverted],
- 2) The 12-bit address of the command within the keyboard monitor and integrated editor as loaded into memory.

The entries repeat until an internal terminator past the last command entry. It is possible to modify the table to prevent certain commands if desired; all standard systems allow all commands per se. [Note: The ED{it} command is inhibited during BATCH operations only.] There is no defined order to the implemented commands; for the most part, they are implemented in alphabetical order.

while not necessarily ever implemented, various command name changes have been considered for the P?S/8 keyboard monitor and integrated line editor. Some of these ideas and related issues follow:

- 1) The PA{use} command was first used in the R-L Monitor System. Since this is in conflict with [at the very least] the PAL program for concise program execution, the command was changed to PL{ease} first in DIBOL-8, COS-300/310 and then P?S/8 some time after the earliest release of COS-300. As a result, practical usage of the command is to embellish it to PLEASE which represents a slight difference in the scope of the usage of the command to alert the operator to some issue that must be resolved immediately. The ^^ command [which does nothing but output the entire command line unless echo is turned off] is often used as a follow-up command:

```
.PLEASE ENABLE THE LINE PRINTER FOR SUBSEQUENT ASSEMBLIES. [The operator presses any key on the system console terminal keyboard and system operation continues, generally in BATCH mode.]
```

```
.^^ THANK YOU
```

- 2) The SE{quence} command was first used in the R-L Monitor System; this usage remained until the 1976 releases of P?S/8. Since this is in conflict with utility programs such as SET [which in the 1976 release of P?S/8 is known as VARY], the command was changed in DIBOL-8 and COS-300/310 to LN{umber}.

Appendix B [Continued].

P?S/8 was released in several stages of transition. The 1976 release of P?S/8 still supports the SE{quence} command; however a short-lived NU{mber} command was also present with the same functionality. Newer releases of P?S/8 deleted the SE{quence} and NU{mber} commands in favor of the LN{umber} command. [Note: It is conceivable that tentative information about the command change in DIBOL-8 was subsequently corrected to LN[umber] however, the information was unofficially communicated to the designers of P?S/8 as NU{mber}. As such, the original SE{quence} command was not removed at that time until better documentation was available which shows the actual command is LN{umber}. When DIBOL-8 was first released, it was difficult to obtain documentation; the product was rushed to market.

Note: It is well-known fact that DEC management was unaware that the DIBOL-8 system was actually a hacked-up version of the R-L Monitor System, a product they had already rejected when offered to them gratis; they simply could not comprehend that this was a complete operating system at the time far better than most of their other software. In future years, P?S/8 was also offered to DEC management for the "price" of permanent equipment donation to the Brooklyn Polytechnic Institute [now a division of NYU] [which DEC could have easily converted into a valuable writeoff perhaps leading to a net profit; certain equipment was **loaned** to the school; however, negotiations broke off with regard to making the computer at the heart of the matter [an 8K PDP-12] become permanently the property of the school [despite the fact that this system was near the end of its economic lifetime and would have to be sold as used equipment]. P?S/8 was shown to DEC personnel twice; each time, indifferent managers rejected it despite massive approval by the members of the PDP-8 programming staff.

- 3) In the interest of future system programs that might be named RESOUR{CE} or similar, the RE{sequence} command could be renamed to the LR{esequence} command which would provide a sense of symmetry with the LN{umber} command.
- 4) The R-L Monitor System uses the MO{nitor} command to restart the system at location 07600 [which is the PDP-8 standard address for rebooting any operating system]. Any of a number of conforming operating systems can be mounted just before entering the command to reboot to another system.

Note: The R-L Monitor System only runs on TC01/TC08/TU55/TU56 DECTape hardware; when the R-L Monitor System was implemented, the other relevant operating systems for this configuration were the DECTape Library System for TC01/TC08 and POLY BASIC [which was developed on the R-L Monitor System]. Due to its inept design, the Disk Monitor System on DECTape cannot be booted via this method; a somewhat longer bootstrap program is required which is incompatible with the R-L Monitor's system kernel.

Appendix B [Continued].

OS/8 and COS-300/310 were released several years after the R-L Monitor System [and early experimental versions of P?S/8]; these systems are also compatible with this restart convention with the additional requirement of extended memory [at least 8K memory is needed except in certain configurations that require 12K memory]. In the earliest stages of development, the forerunner to OS/8 [and PS/8] was booted using the R-L Monitor System's MO{nitor} command; the R-L Monitor System was used to make patches before attempting additional test cycles.

When POLY BASIC [later known as EDU-SYSTEM-30 and sold by DEC] was developed, the notion of leaving BASIC for an underlying [or alternate] operating system was provided by the SY{stem} command. By the use of the SY{stem} command, any of these operating systems for compatible hardware can be booted. Thus, an alternate notion for the command to initiate the restart operation was considered for P?S/8 as the SY{stem} command to replace the MO{nitor} command which would also simplify migrating between systems.

A related issue is the P?S/8 ED[it] command which is patterned after the FOCAL, 1969 MO{dify} command. If the MO{nitor} command was removed, the editing function could be called MO{dify} for consistency with FOCAL, 1969. None of these changes were made in 1976 and the matter was deferred in favor of an unrelated issue. [See 5) below.]

- 5) When the concept of logical units was added to P?S/8, it became clear the syntax to implement this could be added to many commands that would be appropriate. For example, the CA{talog} command could implement a trailing :u where u was a logical unit in the range of 0 through 7. However, this would then require performing an explicit operation; merely changing the logical unit without otherwise performing some operation was not available.

As such, the first implementation of a command to change the logical device unit [and perform no other function] was implemented in early P?S/8 releases as SY{stem}:u. Since the :u syntax is optional, the command SY{stem} without a logical unit specification merely reloads the current TFS directory [or catalog as it often referred to].

It became clear that the SY{stem} command would be in conflict with future programs such as the SYSTAT{us} utility program that provides various system configuration values. With the release of P?S/8 Version 8Y, the command to merely load the TFS directory from a new logical unit was changed to the LC{atalog}:u command which allowed the ability for programs that start with SY to be developed.

Appendix B [Continued].

This interim release also included a transitional version of the SYSTAT{us} utility program. [Note: As of the release of P?S/8 Version 8Z, SYSTAT{us} is mostly functional; however, it includes primitive support for various internal system functions that are subject to change or complete removal in light of the P?S/8 SHELL overlay currently under development.]

Thus, there will be no additional consideration of a command to reload the operating system to be named SY{stem}. As of P?S/8 Version 8Z the MO{nitor} command is used to reload the operating system. The intra-line editing function of the integrated line editor is presently invoked with the ED{it} command and not likely to be changed in the future. [Note: A SYSTEM program utility could be written perhaps to be a more general bootstrap program to support startup from multiple devices; however, it is more likely such a program would be named BOOT with various command-line options as required.]

- 6) Historically, the overall size of computer memory was referred to as the *core size*; the earliest memory implementations as used in most PDP-8 models were measured by this method [such as 4K core]. As such, the P?S/8 program to control the logical size of memory is known as the CORE utility.

It is feasible to change the program name to MEMORY if necessary; however, the original program name applies as of P?S/8 Version 8Z.

If the keyboard monitor is ever modified to include an additional command, it is conceivable it could be known as the CO{py} command. If this is ever implemented, the name of the memory size management utility could be changed to MEMORY or other non-conflicting name.

If additional programs are written to add this functionality [without adding a command to the keyboard monitor] any of these methods would be feasible. A COPY program utility would have its own command syntax, presumably to implement a variety of file copy functions. [Note: P?S/8 Version 8Z includes the FILMAN utility to perform several file-related functions including selectable copy and delete. If desired, the notion of a COPY program could be aliased to the FILMAN program by implementing appropriate command-line switches to force the operating mode.

Appendix B [Continued].

[Note: P?S/8 already supports several aliased programs. The START program is actually an alias within the BIN utility program; the GET program is actually another alias within the BIN utility program. The BIN utility program loads P?S/8 *slurp* binary files or performs various paper-tape-oriented utility functions [including loading in a version of the stand-alone DEC binary paper-tape **BIN** and **RIM** loaders] depending on various command-line options. The START utility program starts up the latest saved memory image and shares certain command-line options with the BIN program. The GET utility program is equivalent to invoking the BIN program with certain options forced for the purpose of loading and saving an image of memory after the loading is completed. The three commands are inter-related; all are actually different entries into the same system program.]

- 7) In early releases of P?S/8, the command to remove a selected range of lines was implemented as the DE{lete} [*line-number-range*] command, which is consistent with the R-L Monitor System. This was changed in Version 8Y to ER{ase} [*line-number-range*] for greater compatibility with DIBOL and COS-300/310. As discussed in 6) above, an additional entry could be added to the FILMAN utility to add a DELETE program function to take advantage of the various options available in the FILMAN utility [such as wild-card matches or file filters]. This functionality is already available in FILMAN, but a DELETE program entry would allow an alternate way to approach command file management functions.
- 8) The complexity of the FILMAN program should allow a file move capability. As such, the issues surrounding the MO{nitor} command may be reconsidered; there would be the need to implement a MOVE alias entry to the FILMAN program; this would necessitate changing the system restart function to a different command. Tentative replacements could include the EX{it} [or EX{odus}] command or perhaps the BY{e} command.
- 9) Due to the trade-offs of supporting high-speed paper-tape devices versus the ability to support more important features, the former HR{eader} and PU{nch} commands were removed from P?S/8 in the Version 8U release. The functions needed are best served in a utility program outside of the scope of the keyboard monitor and integrated line editor. As of this writing, the *PTUTIL* program is being implemented for release as an add-on to P?S/8 Version 8Z. For more information, consult the *PTUTIL Technical Guide* as it becomes available.

Appendix B [Continued].

- 10) P?S/8 supports a wide variety of console terminals, some of which have limited throughput when compared to higher-speed devices. As such, the needs of users of Teletypes and related devices must be served carefully; always outputting the best format cosmetically speaking severely shortchanges users who are merely interested in a rough listing of the contents of a file in a no-frills manner in the shortest amount of time. As such, a stripped-down version of the original PU[nch] command was retained in later releases known as the LT{ext} command. The line number and <HT> character associated with each file line are omitted; only the text characters of the line are sent to the system console terminal. Using the TA{b} 0 command, an unadorned raw version of the text can be obtained in the shortest possible time.

while there are no plans to change this command, another command name considered was the LC{ontents} command to emphasise that only the contents of the lines are printed, not the line numbers. As is always the case, the design goal is to provide an intuitive command name that is also not in conflict with program names that can be as long as six characters. With DIBOL-8/COS-300/310 compatibility already achieved, other command name changes are always possible to deal with design elements unique to the P?S/8 environment. [Note: The P?S/8 SHELL will not have an integrated editor; the design issues are markedly different.]

- 11) with the introduction of the FILMAN utility in P?S/8 Version 8Y, using aliased entry techniques [as practiced in other parts of the basic P?S/8 system] could lead to a more general set of command capabilities somewhat similar to an implementation of CCL. System program such as MOVE, COPY, RENAME and DELETE could be added to take advantage of the filtering and wild-card capabilities of FILMAN. [Note: This technique would be analogous to that used within the DIRECT utility, also introduced in P?S/8 Version 8Y. Since these commands are actually program names, exact names must be used unless six characters long; a true CCL implementation would allow shorter commands as long as the commands were unique.]

As part of any design, all contemplated commands must not be in conflict with existing keyboard monitor and integrated editor commands. [Note: when active development of P?S/8 was put on hiatus with the release of Version 8Z, these file-oriented commands were an open issue regarding future implementation. Since the P?S/8 SHELL overlay will address many of the same issues for an entirely new environment, a byproduct of this new development effort may lead to implementation of the commands as discussed here.]

Appendix B [Continued].

- 12) The TY{pe} command was used in the R-L Monitor System to output the contents of a file to the system console terminal. This command was also used in the 1976 release of P?S/8 Version 8P. For increased compatibility with DIBOL-8 and COS-300/310, the command was changed to the LI{st} command [which is reminiscent of various BASIC implementations which use the same command].

However, in many **CCL**-oriented command processors used in other operating systems, LI{st} is associated with directing output to the system line printer. This issue will be addressed in the design and implementation of the P?S/8 SHELL overlay. It is conceivable that the design intention in COS-300/310 was to use a single command for both operations with an option switch [/T] to force output to the system console printer; this form of command syntax is incompatible with the OS/8 implementation of CCL which is unexpected. The P?S/8 SHELL will create its own set of commands to serve the widest possible user community. [Slavish compatibility with any one system is not the primary goal; improved utility and performance over other PDP-8 operating systems is the main concern.]

- 13) The SE{quence} command was changed to the LN{umber} command for increased compatibility with DIBOL-8 and COS-300/310. An immediate improvement was obtained; the program used to alter many keyboard monitor [and related] settings had been forced to be named VARY because this implementation had been deferred; the utility was renamed in P?S/8 Version 8U to SET.
- 14) At various points in the P?S/8 development effort, a consideration was raised as to whether the RU{n} command would have distinctive meaning from the R command.

Some of these distinctions have to do with the level of implementation of extended-length files. At one point, an entire extended-length file directory structure was completely designed; however, only manual entry of a single file was ever implemented.

The manner in which the present R and RU{n} commands are implemented merely causes some additional swap overhead over the compact subset of **CCL** available in newer of P?S/8; this allows additional entries in the system programs directory where the ability to execute a program by the use of the compact execution form is either superfluous or not possible due to potential anomaly with keyboard monitor and integrated line editor commands. No aspect of the in-memory environment is lost with the possible exception of the edit buffer. Thus, these commands are only required to access programs in the system programs directory which are unrelated to the keyboard monitor environment [such as device-specific formatting and utility programs].

Appendix B [Continued].

Moreover, these commands must be avoided for successful use of keyboard monitor extension utilities such as FIND and CHANGE [which are only useful when used with the edit buffer fully intact].

One possible variant [which was never implemented] was that the scope of the RU{n} command would be increased to allow creation of multiple extended-length files within the extended-length directory. [Note: As of the release of P?S/8 Version 8Z, this structure is still present although inactive.]

With the recent development of the P?S/8 SHELL overlay, all such plans have been abandoned in favor of the far more general variable-length file environment of the SHELL directory structure. As such, the artifacts of the extended-length file directory will be removed. The functionality of allowing one extended-length file per logical unit will be retained; however, the entire directory structure will be replaced with a small reserved area at the end of the system programs directory [beyond all system program entries]. [Note: All execution-class commands require the system programs directory be read into memory to lookup the desired program; as such, the extended-length file entry will also be loaded at the same time reducing overhead as compared with previous releases.]

Appropriate changes to the SET command will be made to allow custom naming of the extended files on each logical unit to suit the user's preference. There will be no operational difference between the R and RU[n] command.

As of this writing, Version 8Z has been released with several binary patches to fix known bugs; all current work is focused on the release of Version 9A some time in the future, which will have the extended-length directory removed as described above.

Presently, only utilities such as the OS8CON utility are capable of writing extended-length files using a work-around as described later in Appendix B. All system programs can access these files for read purposes.

- 15) The LO{ad} command was changed to FE{tch} in subsequent releases of P?S/8 [Version 8U and following] for increased compatibility with DIBOL-8 and COS-300/310. While this does make possible a LOAD alias name for the BIN utility, this was never implemented. The GET alias name is the implemented program name for this function. It is trivial to either change the alias name or allow both variants with identical meaning.

Appendix B [Continued].

- 16) The SA{ve} command was changed to WR{ite} in subsequent releases of P?S/8 [Version 8U and following] for increased compatibility with DIBOL-8 and COS-300/310. It is possible a SAVE utility can be created to transfer the image of memory obtained after a GET program execution to a system program image and execution name; however, there is no guarantee the appropriate storage blocks are set aside on any particular system. As such, this must be deferred until a utility can be written to maintain all other structures further up the system device [such as the starting block of the TFS directory and extended-length file starting block]. [Note: Due to the powerful nature of the P?S/8 system programs directory, a practical SAVE utility must define overlay structures loaded into extended memory if relevant. Program execution may allow operation on a 4K system, but creation of the complete program image will require extended memory unless an alternate facility is created that works with binary files, not the contents of memory.]

There are plans to create a utility with a wider scope to also include the ability to create a working copy of P?S/8 from various binary resources without the need to assemble the source files. Once such a utility completely manages the storage device more effectively, it will be a fairly easy matter to create a SAVE utility as described above.

[Note: Despite the lack of a SAVE utility, P?S/8 currently provides ample facility to deal with core-image programs. P?S/8 ODT operates on the latest memory image after a GET command; from within ODT it is possible to start any program as if it were a system program without restriction. The START utility can pass all parameters with several slight restrictions regarding starting address and memory field. The BSAVE utility can save arbitrary memory locations and memory ranges anywhere in 32K memory into a stream of binary *slurp* files. As of Version 8X of P?S/8, the R and RU{n} commands can run the latest memory image by specifying either % or \$ as the execution name.; the only restriction is the program must start at 00200, the usual starting address of most PDP-8 programs.]

- 17) There were intentions to expand the RU{n} command to fully support the extended-length file directory including multiple variable-length files. Had this been implemented, a distinction would have to be drawn between the R and the RU[n] commands. In addition, the extended RU{n} command would have violated the memory layout of the keyboard monitor regarding the edit buffer and most of the operational code. This would be necessary to load additional support code into memory; a design decision would be required as to whether to allow the edit buffer to be destroyed or alternatively always saving the edit buffer to either the \$ or % positional files. A command-processing error in such an environment would require an entire system reload as well as cause additional overhead.

Appendix B [Continued].

[Note: Independently of these considerations, the SET command supports enabling automatic saving of the edit buffer into the \$ file on the system device and a companion option to load the editing buff from the \$ file on the system device upon any execution-class command; neither of these options are mandatory and one does not depend on the other in any way.]

The original objective was the RU{n} command could create extended-length files that once closed could be read by the R command as it is presently implemented.

with the development of the P?S/8 SHELL overlay, this entire line of system development has been abandoned; the SHELL will implement a significantly more functional environment [which can be enabled or disabled with a single command]. As appropriate, conversion programs will be provided to migrate extended-length files to the SHELL environment,

- 18) Due to the nature of the modifications regarding support for the new definition of the extended-length file [with only a single extended-length file per logical device unit], a temporary restriction exists with regard to access to the extended-length file until these changes are implemented in the next release. A viable workaround procedure will be presented.

Extended-length files are created by programs such as OS8CON and L6DCON when appropriate options are used to distinguish extended-length output file format from the that used with TFS output files.

When fully implemented, a level of safety will be implemented to ensure passed output files are actually in the extended-length file area; however, for the present these conversion programs do not check for this. This was made necessary because it is not possible to specify extended-length output files presently.

The user is advised to create extended-length files carefully by specifying the highest block file within the TFS directory. [Note: When the actual highest possible TFS file block is used for this purpose, the value is exactly 16 blocks before the extended-length entry in the proper directory format would be. As such, as long as the user ensures the proper placement for the target TFS output files, these utilities can be used with impunity.]

When the next release is completed, the storage device can be configured for proper block utilization; extended files created using the workaround can be converted with OS8CON and then converted back to proper blocks. [Note: The restriction does not affect the operation of OS8CON with regard to converting P?S/8 files to OS/8 data.]

Appendix B [Continued].

Extended-length output file workaround example.

Console output	Comments
.	Keyboard monitor prompt.
.CA	CA{talog} command.
FLNAM1 FLNAM2 FLNAM3 FLNAM4	Contents of TFS directory.
	Highest file name is FLNAM4.
.R OS8CON,FLNAM4</1=7 /E	Run OS8CON with the highest file name as output.
OS8CON V10b	Program announcement including version and revision.
OS/8 => P?S/8 [Extended File]	Extended file creation because the /E option was given.
1 File Created	File creation report. One extended-length file created.
.	Keyboard monitor prompt.

In the above example, the highest file present in the directory was used to locate the extended-length file. This is acceptable as long as no additional TFS files are created past this file. To minimize the danger of file corruption, all available slots in the TFS directory should be assigned to files; the highest slot available will then reference output blocks starting 16 blocks before those intended for the extended-length file once the workaround is no longer needed.

Once the highest slot is assigned to the file to be used for extended-length file creation, any dummy entries created to position the highest file may be deleted [if desired]. [Note: when the OS8CON and L6DCON programs are updated to ensure extended-length file creation is only applied to legitimate extended-length files, the workaround will cease to be functional.]

End of Appendix B.

Appendix C.

Seven-bit ASCII character usage in the P?S/8 keyboard monitor and related system components. [See Note 1.]

<u>Seven-Bit ASCII Code</u>			<u>Func</u>	<u>Alt</u>	<u>P?S/8</u>	<u>Remarks</u>
<u>Dec</u>	<u>Octal</u>	<u>Hex</u>			(Six-bit Octal)	
Section 1: Non-printing characters.						
0	000	00	<NUL>	<BRK>	n/a	See Notes 2, 3.
1	001	01	<SOH>	<^A>	n/a	See Notes 3, 4.
2	002	02	<STX>	<^B>	n/a	See Note 3.
3	003	03	<ETX>	<^C>	n/a	See Note 3.
4	004	04	<EOT>	<^D>	n/a	See Note 5.
5	005	05	<ENQ>	<^E>	n/a	See Note 6.
6	006	06	<ACK>	<^F>	n/a	See Note 7.
7	007	07	<BEL>	<^G>	n/a	See Note 8.
8	010	08	<BS>	<^H>	n/a	See Note 9.
9	011	09	<HT>	<^I>	37	See Note 10.
10	012	0A	<LF>	<^J>	n/a	See Note 11.
11	013	0B	<VT>	<^K>	n/a	See Note 12.
12	014	0C	<FF>	<^L>	n/a	See Note 13.
13	015	0D	<CR>	<^M>	00	See Note 14.
14	016	0E	<SO>	<^N>	n/a	See Note 15.
15	017	0F	<SI>	<^O>	n/a	See Notes 15 and 16.
16	020	10	<DLE>	<^P>	n/a	See Note 17.
17	021	11	<DC1>	<^Q>	n/a	See Note 18.
18	022	12	<DC2>	<^R>	n/a	See Note 19.
19	023	13	<DC3>	<^S>	n/a	See Note 18.
20	024	14	<DC4>	<^T>	n/a	See Note 19.
21	025	15	<NAK>	<^U>	n/a	See Note 20.
22	026	16	<SYN>	<^V>	n/a	Ignored.
23	027	17	<ETB>	<^W>	n/a	Ignored.
24	030	18	<CAN>	<^X>	n/a	Ignored.
25	031	19	<EOM>	<^Y>	n/a	Ignored.
26	032	1A	<SUB>	<^Z>	n/a	See Note 5.
27	033	1B	<ESC>	<ALT>	n/a	See Note 21.
28	034	1C	<FS>		n/a	Ignored.
29	035	1D	<GS>		n/a	Ignored.
30	036	1E	<RS>		n/a	Ignored.
31	037	1F	<US>		n/a	Ignored.

Appendix C [Continued].

<u>Seven-Bit ASCII Code</u>			<u>Funct</u>	<u>Alt</u>	<u>P?S/8</u>	<u>Remarks</u>
<u>Dec</u>	<u>Octal</u>	<u>Hex</u>			(Six-bit Octal)	

Section 2: Non-alphabetic printing characters.

32	040	20	<SP>		40	
33	041	21	!		41	
34	042	22	"		42	
35	043	23	#		43	
36	044	24	\$		44	
37	045	25	%		45	
38	046	26	&		46	
39	047	27	'		47	
40	050	28	(		50	
41	051	29	)		51	
42	052	2A	*		52	
43	053	2B	+		53	
44	054	2C	,		54	
45	055	2D	-		55	
46	056	2E	.		56	
47	057	2F	/		57	
48	060	30	0		60	
49	061	31	1		61	
50	062	32	2		62	
51	063	33	3		63	
52	064	34	4		64	
53	065	35	5		65	
54	066	36	6		66	
55	067	37	7		67	
56	070	38	8		70	
57	071	39	9		71	
58	072	3A	:		72	
59	073	3B	;		73	
60	074	3C	<		74	
61	075	3D	=		75	
62	076	3E	>		76	
63	077	3F	?		77	

Appendix C [Continued].

<u>Seven-Bit ASCII Code</u>			<u>Funct</u>	<u>Alt</u>	<u>P?S/8</u>	<u>Remarks</u>
<u>Dec</u>	<u>Octal</u>	<u>Hex</u>			(Six-bit Octal)	
Section 3: Upper-case alphabetic printing characters.						
64	100	40	@		n/a	See Note 22.
65	101	41	A		01	
66	102	42	B		02	
67	103	43	C		03	
68	104	44	D		04	
69	105	45	E		05	
70	106	46	F		06	
71	107	47	G		07	
72	110	48	H		10	
73	111	49	I		11	
74	112	4A	J		12	
75	113	4B	K		13	
76	114	4C	L		14	
77	115	4D	M		15	
78	116	4E	N		16	
79	117	4F	O		17	
80	120	50	P		20	
81	121	51	Q		21	
82	122	52	R		22	
83	123	53	S		23	
84	124	54	T		24	
85	125	55	U		25	
86	126	56	V		26	
87	127	57	W		27	
88	130	58	X		30	
89	131	59	Y		31	
90	132	5A	Z		32	
91	133	5B	[		33	
92	134	5C	\		34	
93	135	5D	]		35	
94	136	5E	^		36	
95	137	5F	_		n/a	See Note 23.

Appendix C [Continued]..

<u>Seven-Bit ASCII Code</u>			<u>Funct</u>	<u>Alt</u>	<u>P?S/8</u>	<u>Remarks</u>
<u>Dec</u>	<u>Octal</u>	<u>Hex</u>			(Six-bit Octal)	
Section 4: Lower-case alphabetic printing characters.						
96	140	60	`		n/a	See Note 22.
97	141	61	a		01	See Note 24.
98	142	62	b		02	See Note 24.
99	143	63	c		03	See Note 24.
100	144	64	d		04	See Note 24.
101	145	65	e		05	See Note 24.
102	146	66	f		06	See Note 24.
103	147	67	g		07	See Note 24.
104	150	68	h		10	See Note 24.
105	151	69	i		11	See Note 24.
106	152	6A	j		12	See Note 24.
107	153	6B	k		13	See Note 24.
108	154	6C	l		14	See Note 24.
109	155	6D	m		15	See Note 24.
110	156	6E	n		16	See Note 24.
111	157	6F	o		17	See Note 24.
112	160	70	p		20	See Note 24.
113	161	71	q		21	See Note 24.
114	162	72	r		22	See Note 24.
115	163	73	s		23	See Note 24.
116	164	74	t		24	See Note 24.
117	165	75	u		25	See Note 24.
118	166	76	v		26	See Note 24.
119	167	77	w		27	See Note 24.
120	170	78	x		30	See Note 24.
121	171	79	y		31	See Note 24.
122	172	7A	z		32	See Note 24.
123	173	7B	{		33	See Note 24.
124	174	7C			34	See Note 24.
125	175	7D	}		35	See Notes 24, 25.
126	176	7E	~		36	See Notes 24, 25.
127	177	7F	<RO>		n/a	See Note 9.

Appendix C Notes.

- 1) Certain terminals are capable of generating an eight-bit character code, especially Teletype models 33 ASR and 35 ASR, which have attached paper-tape readers. While the high-order bit of the eight-bit data might have meaning in the paper-tape system, P?S/8 universally strips off the high-order bit; as such, there are only seven significant bits in the character code set for consideration.

In all contexts, the parity bit is ignored. [Note: Various P?S/8 utilities are designed to import/export from/to paper-tape devices; where applicable, the parity bit will always be set to conform to paper-tape mark parity conventions.]

Appendix C Notes [Continued].

- 2) `<NUL>` [also known as `<BRK>`] and `<RO>` [also known as `<DEL>`] are used as part of timeout schemes for various output devices such as the standard device 66 line printer or device 02 high-speed punch. These optional devices may not be present in the current configuration; alternatively, the operator may have placed one of the devices in an offline state to temporarily prevent access. While no one method applies in all circumstances, these codes are generally chosen as part of the device initialization; the outputted data is usually ignored.
- 3) Many PDP-8 programs support some form of system console keyboard-based forced exit strategy. The most widely used exit code is `<ETX>` [also known as `Control-C`] which is often printed as `^C` by the program's exit monitoring routines or the operating system upon detection.

Within the P/S/8 operating system, the system BATCH facility will abort the current sequence if `Control-C` is detected. However, if a program exits to the operating system using the standard restart address of 07600, BATCH abort may be avoided by the use of alternate exit characters. Many P/S/8 system programs check for the additional codes `<NUL>` [also known as `<BRK>`], `<SOH>` [also known as `Control-A` and sometimes printed as `^A`] and `<STX>` [also known as `Control-B` and sometimes printed as `^B`]. Since the P/S/8 keyboard monitor checks explicitly for `Control-C`, the latest program can exit without aborting the overall BATCH operation that may be in effect.

- 4) `<SOH>` [also known as `Control-A`] is used within the keyboard monitor `ED{it}` command to initiate an additional search for the character located in the previous search operation.
- 5) `<EOT>` [also known as `Control-D`] was used to end auto-sequenced text input in the P/S/8 keyboard monitor prior to Version 8Y. In newer releases, this function is performed by `<SUB>` [also known as `Control-Z`] to conform with existing industry standards for end of medium.
- 6) `<ENQ>` [also known as `Control-E`] is not used in P/S/8. When running P/S/8 from the SIMH PDP-8 simulator, `Control-E` is used to exit the simulator after closing all open devices.
- 7) `<ACK>` [also known as `Control-F`] is used within the keyboard monitor `ED{it}` command to print out any characters remaining in the buffer to allow either further editing or closing out the line being edited.

Appendix C Notes [Continued].

- 8) `<BEL>` [also known as `Control-G`] is used within the keyboard monitor `ED{it}` command to initiate a search for a character in the buffer. All characters up to and including a match between the next character typed and the remaining contents of the buffer will be displayed; further editing operations are allowed including additional search operations.

Each time the keyboard monitor is reloaded, a `<BEL>` character is sent to the system console terminal to indicate the system is ready for further keyboard input. Future releases of the P?S/8 SET utility will include the command `SET SYSTEM [NO] BEEP` to manage this action.

- 9) within the keyboard monitor, the *rubout* [delete previous character] function can be initiated by several characters including the `<BS>` character on the system console keyboard [if present] and the `<RO>` key [often labeled `RUBOUT`]. When the PDP-8 is being simulated on various PC systems, there is often an enlarged key with a legend consisting of a left-facing arrow [often labeled `Backspace`]. The actual code generated within the simulated environment could be either `<BS>` or `<RO>`; most PDP-8 programs handle either code as initiating the *rubout* function as described above.

The `<BS>` character is often used on video terminals as part of the support for the the *rubout* function. Most video terminals will visually delete and backup the cursor if the sequence `<BS> <SP> <BS>` is sent to the terminal. However, certain devices such as the VT05, VT52 and VT100 are supported using custom character sequences particular to each terminal. Additional information about the P?S/8 SET utility is available in other documents.

- 10) `<HT>` is used to align text to boundaries of eight columns in many DEC systems as well as MS-DOS, Unix and windows. The specific interpretation of the `<HT>` character in the keyboard monitor is determined by the `TA{b}` command which can be set to accommodate a wide variety of system console terminals. The default setting is used for support of the `<HT>` on devices such as the Teletype Model 33 which does not support the `<HT>` character directly; `<SP>` characters are output as necessary to obtain the desired visual alignment. The Teletype Model 35 is supported using another setting particular to this device; generic support is available for devices that support the `<HT>` directly. For rough draft output on very slow devices, `<HT>` support can be reduced to always printing a single `<SP>` character.

For systems that are always configured for a specific terminal, the SET utility can change the default setting to any of the supported modes. Additional information about the P?S/8 SET utility is available in other documents.

Appendix C Notes [Continued].

- 11) The `<LF>` [also known as `Control-J`] character is used as part of the `<CR>/<LF>` sequence required on all printed output and supported by all system programs when outputting to the system console terminal or system line printer.

within the keyboard monitor, pressing `<LF>` will output `<CR>/<LF>` followed by the tentative contents of the command-line buffer. This is useful on hard copy terminals especially after multiple rubout operations have occurred; a clear view of the tentative command-line is often needed to better understand the command in progress.

When paper-tapes of text data are read in on the readers of such devices as the Teletype Model 33 ASR and Model 35 ASR, the text often includes both `<CR>` and `<LF>` for each line of text; as such, it is undesirable to allow the usual operation of the `<LF>` during auto-sequenced text input operations as this will tend to produce undesirable doubled output. To prevent this, use the system program command `SET LNFEED IGNORE`; the normal function of echoing the current tentative command line can be restored by using the system program command `SET LNFEED VIEW`. Information about additional P?S/8 SET utility options is available in other documents.

- 12) `<VT>` [also known as `Control-K`] is used within the keyboard monitor `ED{it}` command to delete all characters in the command-line buffer already processed and sent to the system console printer.
- 13) The `<FF>` character [also known as `Control-L`] is used on the system line printer to separate physical pages of output; all printer must support this function properly.

P?S/8 text files do not support the `<FF>` character directly; however, P?S/8 PAL supports the `EJECT` directive which will output a `<FF>` causing further output to be placed on the next page. [The statement which includes the `EJECT` directive and an optional argument is not displayed within the assembly listing. The optional argument, if present, is used to change the title of the assembly listing on the next printed page in a manner similar to the `TITLE` directive.]

The prevailing conversion programs [such as P?S/8 `OS8CON`] will convert the character sequence `<HT> EJECT <CR>` to `<FF>` for use in systems that support `<FF>` directly. [Support of this function is available in both directions of the conversion process.]

- 14) The `<CR>` character [also known as `Control-M`] is used to terminate all keyboard monitor commands. Commands that insert text lines into the edit buffer terminate the line internally with 00.

Appendix C Notes [Continued].

There is an alignment consideration when terminating any text line: If the `<CR>` is being applied by inserting 00 into the low-order bits of the latest 12-bit word, no further changes are required. If the `<CR>` is being applied by inserting 00 into the high-order 12-bits of the latest word, the low-order bits must be set to a non-zero value to avoid confusion with the file text terminator, which is a full 12-bit word of 0000.

- 15) `<SO>` [also known as `Control-N`] and `<SI>` [also known as `Control-O`] are used on certain devices [such as the LA36 DECwriter with the APL option installed] to change the operating mode from ASCII to the APL character set. Optional keycaps are used on terminals that support this option to assist the user as the two operating modes are significantly different. [The DataSouth DS-120 speedup board for the LA36 always includes APL support.]
- 16) `<SI>` [also known as `Control-O`] is used in many P?S/8 printing programs to invert the state of printing; this allows voluminous output to be temporarily ignored; when `Control-O` is pressed a second time, the output will continue [generally much further along in the overall printing process]. However, it takes practice to determine how long to wait between key pressings of `Control-O` to avoid total loss of the rest of the output. The primary purpose of the `Control-O` function is to effectively cancel assembly listing accidentally sent to the system console instead of the system line printer.
- 17) `<DLE>` [also known as `Control-P`] is used in certain P?S/8 utility programs to restart the utility generally abandoning the previous command or operation.
- 18) `<DC3>` [also known as `Control-S`] and `<DC1>` [also known as `Control-Q`] are used to pause and release all P?S/8 system console output as part of the industry standard "XON/XOFF" protocol. Many DEC and DEC-compatible devices such as the VT-100 and LA-120 require this protocol for proper operation. Various serial line printer devices which use the XON/XOFF protocol are also supported by the P?S/8 Logical Console Overlay [when suitably configured].

Very few operating systems for the PDP-8 fully support XON/XOFF protocol. All members of the DEC OS/8 family of operating systems are incapable of this support at the software level with the exception of partial support when output is directed to a specific handler [generally known as KL8E]. However, all OS/8 programs perform their own console handling completely devoid of XON/XOFF support leading to garbled output on affected devices unless all output is reduced to very low baud rates.

Appendix C Notes [Continued].

Instead of redesigning OS/8, DECmate models were released with the ability of the internal terminal emulator to prevent buffer overrun without the use of any protocol; however, OS/8 family members [such as OS/278] cannot provide support for the PAUSE key or any other serial console device without the same problems as all other similar systems.

- 19) `<DC2>` [also known as `Control-R`] and `<DC4>` [also known as `Control-T`] are used respectively to rewind and unload the current system device. Support is hardware-dependent; both commands may perform the same operation or no operation for certain hardware configurations.

On the P?S/8 system for TC01/TC08/TU55/TU56, pressing `Control-R` while entering any command or other keyboard input on the system console device will rewind the current system device unit. Control is immediately returned to the keyboard monitor; as such, commands may be typed while the DECTape is rewinding. This can be useful when an execution-class command is being entered, as the DECTape blocks required to process the command are located near the beginning of the DECTape. The tape is automatically stopped by the DECTape controller when the reverse end zone is detected.

Pressing `Control-T` on the same configuration will rewind the system device unit; however, the DECTape status is continuously monitored until the end zone is detected. Additional commands are issued to force the DECTape off the takeup reel. If the DECTape drive is a TU55, the brakes are locked stopping the motors; TU56 drives will require manual intervention by placing the drive off-line.

- 20) `<NAK>` [also known as `Control-U`] is used to cancel the latest command or other input to the keyboard monitor. When used during the keyboard monitor `ED{it}` command, `Control-U` completely restarts the editing process with the original line contents.

When used in auto-sequenced text input mode, the latest line number is repeated; all input on the current line is discarded.

- 21) `<ESC>` [also known as `<ALT>`] is used to terminate all input to the keyboard monitor and in most contexts is equivalent to typing `<CR>` with the following exceptions:
- a) Typing `<CR>` will terminate the line and output `<CR>/<LF>` to start another line in the usual manner that applies. Typing `<ESC>` will output `$` only.

Appendix C Notes [Continued].

- b) Certain programs are designed to scan the command line by setting special scanning restrictions within the keyboard monitor. When the command line is terminated by typing `<ESC>` the program may change its operating mode [generally to imply program exit after performing the specific command passed on the command line]. Such usage only applies to programs designed to scan the command line and operate either repeatedly or once-only depending on the command line termination character.

On certain terminals, the `<ESC>` function may be provided by a key marked as `ALT MODE` [or perhaps `ALT`]. The actual character code generated by pressing this key might be either 175 octal or 176 octal depending on exact terminal characteristics. The disposition of the `<ESC>` function and the exact keyboard code used on any particular console device can be configured with the P?S/8 SET utility. Additional information about the P?S/8 SET utility is available in other documents.

- 22) P?S/8 text files do not support `@` or ``` [the latter is supported in some systems as the lower-case equivalent of the former].

In the full definition of the PDP-8 Assembly Language [PAL] specifications, the only valid use of the `@` character is as a variant delimiter character within TEXT statements; however, many other characters are available to serve the same purpose. The only requirement of a delimiter character is to frame the text string consisting of valid text characters other than the delimiter character; as such, there should never be any problematic cases of TEXT statements that cannot be stated using one of the other valid delimiter characters.

Recent releases of P?S/8 PAL support this ability to use ordinary printing characters as delimiter characters in TEXT statements. As such, the problem of delimiter substitution is somewhat easier to solve than in previous releases.

Since no implementation of the PAL language allows lower-case input within statements, the ``` is not a valid delimiter of TEXT statements.

The latest version of the inter-system file conversion programs such as OS8CON, *L6DCON* and the soon to be released PTUTIL utility will implement conversion of both the `@` and ``` characters to the `#` character. This is an acceptable substitution since the only allowed usage for these characters could be as text string delimiters such as are used with the TEXT directive and related assembler features. [Note: In the event this is in conflict with the text string contents, manual editing will be required.]

Appendix C Notes [Continued].

Note: The P?S/8 SHELL will support an advanced assembler based on P?S/8 PAL [tentatively named XPAL] which will support the seven-bit character set. Character set rules of this implementation will be greatly expanded compared to all other known versions of PAL.

- 23) When FOCAL, 1969 was designed, the displayed character associated with ASCII code 95 decimal [137 octal] [5F hex] was implemented on Teletype Model 33 and similar console terminals as ; FOCAL and various other programs on other systems used this character code to logically delete all characters on the current command line or equivalent. The displayed character was changed to  which was no longer considered appropriate for representing functions of this nature. As such, more recent practice is to consider this value as one of the supported codes that initiate the *rubout* function as described in Note 9 above.

The latest version of the inter-system file conversion programs such as OS8CON, *L6DCON* and the soon to be released PTUTIL utility will implement conversion of this character to the - character. This is consistent with various industry standards that find the character problematic in certain contexts such as ISO filenames. The user may consider post-conversion editing of any files where this is an issue.

The P?S/8 SHELL is currently under development. The standard character set for the SHELL will be full seven-bit ASCII. As such, _ will be treated as an ordinary printing character; it will not be possible to perform the *rubout* function using this character code.

It is impossible to directly underline printing characters on most video terminals unless there are extended capabilities particular to the terminal. Printing devices can create this effect by the use of a multiple strike routine to overprint the desired character with _ . This is generally carried out in word-processing utilities as a specialized form of text output variation tailored to a specific printer.

The P?S/8 SHELL may implement a text-processing program such as RUNOFF; printout functions such as underlining may only work on certain printing devices which must be tested on a case by case basis.

- 24) All lower-case alphabetic characters are automatically converted to upper-case characters in the keyboard monitor. As such, the P?S/8 text code shown applies to the upper-case equivalent. Prevailing conversion programs such as OS8CON will perform the same case conversion when OS/8 input files are converted to P?S/8 text files.

Appendix C Notes [Continued].

- 25) The character code for the `<ESC>` key is generally 27 decimal [033 octal [1B hex] and typically labeled `ESC` ; however, very early production Teletype Model 33 and Model 35 terminals use a key labeled `ALT MODE` [or `ALT` in some cases]. Certain production models were used on PDP-8 systems where the code generated by pressing this key is 125 decimal [175 octal] [7D hex] or 126 decimal [175 octal] [7E hex].

Various third-party maintenance companies were contracted by certain PDP-8 users unsatisfied with the generally poor quality of DEC field service, especially with regard to Teletype products. Many of the employees of these service companies were experts who had worked for the Teletype corporation before their position as field technicians. Some of them were known to offer custom modifications to these terminals for an additional service fee [or perhaps as a personal service]. As such, virtually every combination of keycap legend and generated code described in this document were to be found on certain of these terminals.

Users of LINC-8 systems who preferred to use PDP-8 software instead of primarily LINC-oriented software often paid to have the entire set of keys replaced as well as the original typewheel which printed the earliest of the various LINC character sets that were supported over the years. DEC supplied keys with legends of very early LINC Assembly Program [LAP] systems while the prevailing software developers were simultaneously changing the character sets displayed on the screen. As such, key legends and typewheel characters could not keep up with which particular LAP-type system was in vogue.

Some of the LAP-type systems included conversion programs to upgrade existing source programs to be compatible with the newer releases. With no central standard to conform to, it was inevitable that replacement keys with different legends would be needed as well as replacement typewheels that were in compliance with the particular LAP system in use. With the expertise of the technicians available, particular users could generally stay in sync with the needs of the software systems they embraced.

Eventually, the newest LAP-type systems embraced the ASCII character set rendering all prior generations of key legends and typewheels obsolete. In the end, users of PDP-8 software and LINC-oriented software generally agreed to use the more typical ASCII key legends and typewheels.

Appendix C Notes [Continued].

For specific users who insisted on obsolescent personal preference, handmade legend strips were created and glued onto the terminal adjacent to the corresponding keys to aid in deciphering the replaced keys to maintain their former meaning; since the LINC-coded typewheel had been converted to an ASCII-coded typewheel, it was impossible to print the former LINC characters [assuming a match to the particulars of the user's preferred LAP version].

When a Teletype Model 33 ASR is encountered as a LINC-8 console terminal, it is possible the key legend for the `<ESC>` key is marked `META` for cosmetic compatibility with the original LINC computer regardless of the actual code generated when the key is pressed.

P?S/8 supports a designated code for `<ESC>` which defaults to 033 octal. The SET utility program can be used to change the designated code to conform to local requirements. As long as the code for `<ESC>` is not customized to 175 octal or 176 octal, these character codes are available to the keyboard monitor for their designated purpose and will be converted to the upper-case equivalent.

The disposition of certain characters as described here is for the basic P?S/8 system as of Version Z. Future releases will include the P?S/8 SHELL overlay; files created in this environment will support nearly the entire seven-bit character set [and perhaps certain extensions].

End of Appendix C.

Glossary of terms used in this document.

%

The % file is located in blocks 0020-0037 on every logical device unit. % and \$ valid alternatives to standard TFS file names and can be accessed more quickly than any other file due to their position near the beginning of the device. There are several sets of commands and system programs that, while consistent with each other in terms of usage of the % and \$ file space, are in conflict with the usage of the % and \$ files in other situations. Examples are given below. Considerable time can be saved by consistent usage of the % and \$ files in any one of the available contexts.

The % file is used in conjunction with the \$ file as the designated virtual core image image of field 0 when working with the latest program using the system ODT debugging program. The START, GET and BSAVE utilities are compatible with this usage of % and \$.

For all other purposes, the % and \$ files can be used as an alternative to a named TFS file; in certain programs [such as PAL, MAP and BIN], option switches exist to use % and \$ as default binary files. Permanent copies of the % and \$ files can be made using appropriate keyboard monitor commands while using the edit buffer as intermediary storage.

\$

The \$ file is located in blocks 0040-0057 on every logical device unit. Most of the considerations of the % file apply with the exception of certain command settings and default file options in certain programs where one of the two files is used instead of the other.

Certain automatic edit buffer saving and/or restoring operations apply to the \$ file. Generally, in situations where the % file is considered the first default binary file, the \$ file is considered the second default binary file [if applicable] as determined by option switches to the affected programs.

BIN

The DEC standard paper-tape format used for most binary file loading from the stand-alone environment defined as the CPU, the system console terminal and a paper-tape reader. The reader may be either a high-speed reader [device code 02] or part of the system console [generally a Teletype Model 33 ASR or Model 35 ASR].

Glossary of terms used in this document [continued].

P?S/8 implements a compatible version of the **BIN** loader that can be loaded into the last page of memory [07600-07777] along with the **RIM** loader as is generally the case with other versions of similar programs.

Note: There are several known implementations of the **BIN** loader, all of which are designed to occupy some portion of 07600-07777; due the differences in exact loading, the starting address is defined as 07777 which is actually a JMP instruction referencing the internal starting address to avoid the need for version-specific documentation.

The P?S/8 implementation adds additional functionality; when the **BIN** and **RIM** loaders are in memory, a standalone bootstrap for the current P?S/8 configuration is loaded into 07600-07642. P?S/8 can be reloaded by starting the computer at the standard restart address of 07600. The only requirements for a successful reload are: a) The data field is reset to 00, b) None of the locations in the range of 07600-07642 are modified.

Note: There are a small number of known "rogue" programs that violate this convention; some can actually destroy specific implementations of the **BIN** loader due to poor design and expectation of the memory layout of a particular implementation of the **BIN** loader which is contrary to all known PDP-8 programming conventions. The only recourse after loading one of these programs is to manually reload memory to repair the program corruption; however, most of these programs do not violate the **RIM** loader memory layout. All implementations of the **RIM** loader are identical; as such, copies of the contents of 07600-07754 can be punched in **RIM** format to restore memory as needed.

All standalone implementations of the **BIN** loader use the front panel switch register to select choice of reader hardware. The P?S/8 implementation of **BIN** and **RIM** includes preconfiguration by the use of option switches; as such, the front panel does not affect either program. [Note: Not all machines support a front panel.] The P?S/8 system program **BIN** [not to be confused with **BIN**] is used to load the standalone bootstrap, the **BIN** loader and the **RIM** loader via the following command:

```
.BIN /P /F /R or (PFR)
```

/P must be used to indicate the paper-tape loaders will be loaded into memory [instead of the contents of *slurp* binary files].

Glossary of terms used in this document [continued].

/F can be optionally specified to indicate the choice of the high-speed reader, which is applied to both **BIN** and **RIM**; if left out the low-speed reader is used in both loaders.

/R can be optionally specified to indicate the **RIM** loader will be started; the default is to start the **BIN** loader.

Core image [or core-image when used as an adjective].

A Core image refers to data stored on blocks of storage where entire pages of program or other data are presented in 128 word units meant to be loaded into various memory locations. Specific loading of groups of core-image blocks is determined outside of the core-image data such as within directory or other complex file structures that manage loading and mapping issues.

Core images stored in order of adjacency by loading address are the most efficient form of direct binary storage for executable programs assuming the operating system maintains relevant loading parameters [such as the P?S/8 system programs directory].

P?S/8 system programs are stored in core-image form for at least the first load segment of fully adjacent storage blocks; additional segments may be defined by the program for additional purposes such as program or data overlays.

Slurp-format binary files are not core-image files; however, *slurp*-format binary files can be loaded into memory and saved into system blocks 0020-0057 which is a core image of all of field 0. Once saved, various utilities exist to interact with the core image saved in these blocks which are also known as the system scratch area. For more information, see the definition of the % and \$ files. Copies of these files can be saved to named TFS files if desired. Restoring the % and \$ files to their former contents will restore the core image to its former state; this allows additional work to be performed on an ongoing project perhaps undergoing debugging with P?S/8 ODT.

While P?S/8 does not currently provide a means to directly create core-image system programs, the current core-image can be run using either the R [or RU{n}] % or R [or RU{n}] \$ command; all of the options of typical P?S/8 system programs are available except for the possibility of internal overlays. A thorough knowledge of P?S/8 is required to create complex system programs on a permanent basis. Only advanced users will be technically qualified to take advantage of the overlay facility inherent in the system programs directory.

Glossary of terms used in this document [continued].

A complete understanding of the internal workings of P?S/8 execution and interaction with the system programs directory and the system device handler are necessary prerequisites for a successful system program project. Advanced information on this topic will be made available in separate documents covering P?S/8 principles of operation and other information. [Note: Once these topics are mastered, the ability to create any form of permanent system program will be trivial and quite obvious; however, future releases of P?S/8 will provide utilities to assist in the system program creation process.]

Edit Buffer

The edit buffer is a 2048 word memory area internal to the keyboard monitor and integrated editor. It is used to temporarily hold a TFS text file during all editing operations. File updates are made permanent by writing the contents of the edit buffer to a named file. If keyboard monitor commands are strictly limited to loading and saving the edit buffer, TFS files of any format can be copied using the edit buffer as intermediary storage.

P?S

The Poly Question Society [P?S] is the self-appointed name of a group of questioners most of whom met at the school affectionately known as Brooklyn Poly or Brooklyn Polytech or the Polytechnic Institute of Brooklyn. Various other names of the school have existed over the years; as of the time of this writing, the school has been taken over by New York University [NYU] and many people have many names for this institution, some of which are not necessarily suitable for publication or even discussion in mixed company.

Most of the members of the P?S [whether living, dead, or mythological creatures] are additionally somewhat legendary in the annals of computer programming [and in some cases beyond the limitations of their own minds] and are responsible for many programming projects, most of which were eventually products of Digital Equipment Corporation [DEC].

When and where possible, P?S should be spelled *P?S* to be pronounced as pee-kew-ess; If necessary, it can be printed as PQS which corresponds to the pronunciation.

The P?S also has an official symbol which closely resembles the large ? as seen on various costumes worn by the late Frank Gorshin as The Riddler on the 1966 TV show *Batman*. In print, the Lucida Console font as used in this document renders the ? character in a fairly acceptable way.

Glossary of terms used in this document [continued].

P?S/8

This document is an attempt at a companion manual for P?S/8, a product written by several P?S members with the assistance of several other computer programmers, some of whom have actually written PDP-8 programs prior to their participation in the project [or since]. [In certain cases, remnants of their participation are actually still considered a part of the overall project; in some cases this includes source code not necessarily commented out which produces some of the binary code that is part of recent releases. In specific cases, the names of these people have been withheld for a variety of reasons, none of which have to do with any known witness protection programs].

P?S/8 should be pronounced as pee-kew-ess-ate; if necessary, it can be spelled as PQS/8 and if applied to portions of related file names, abbreviated to PQS8 in all capital letters.

Virtually all current support for P?S/8 is done by Charles Lasner, who has written much of the code, and is also the author of this document.

RIM

This is the alternate standard DEC format for binary paper-tape loading. **RIM** format is used primarily to load in the **BIN** loader or other short binary program. It is less versatile than **BIN** format; however the loader is much shorter and can be entered manually from the front panel of most PDP-8 systems.

[Note: **RIM** format binary paper-tapes are longer than **BIN** format binary paper-tapes because automatic increment of the loading address is not supported; each memory location must include both an address as well as the contents even if the loading happens to be sequential. Field settings are not supported; as such, program loading is limited to a single memory field. The data field for program loading must be set manually on the front panel. Most importantly, **RIM** format does not support data checksumming. This is the principal reason why **RIM** format is only recommended for loading short programs such as the **BIN** loader.]

Most PDP-8 storage-based operating systems can be started using a bootup program even smaller than the **RIM** loader; as such, given a choice, it is generally less manual effort to reload P?S/8 than to toggle in the **RIM** loader. For certain configurations, the reverse is true; in those instances, the **RIM** loader presents a better choice for computer startup [assuming a paper-tape reader is available].

Glossary of terms used in this document [continued].

Slurp

This is the name given to the P?S/8 binary file format by its inventor, Richard Lary, coauthor of the R-L Monitor System, which is the precursor system P?S/8 is partially based on.

Slurp binary format is the most efficient format for loading random access binary files into memory from a storage device. Actual loading efficiency approaches 5/6 that of block-oriented [core-image] binary format while allowing the loading of individual locations or large blocks of contiguous memory without requiring any particular block structure. Memory can be freely loaded anywhere in the entire PDP-8 32K memory space [other than system reserved memory].

Most configurations of P?S/8 support a standalone *slurp* loader that displaces the system device handler. The standalone loader includes the same system restart code as defined in the description of the P?S/8 variant of the **BIN** loader elsewhere in this document.

The P?S/8 BIN program initiates the *slurp* loader passing a list of *slurp*-format binary files specified by the user for loading using a command such as the following:

.BIN *list of input files* /H /Z /I /V or (HZIV)

/H can be optionally specified to indicate all non-reserved memory will be filled with 7402 [the PDP-8 HLT instruction] before the contents of the input files are loaded.

/Z can be optionally specified to indicate all non-reserved memory will be filled with 0000 [the LINC HLT instruction] before the contents of the input files are loaded. [Note: /H and /Z are mutually exclusive and cannot be used together.]

/I can be optionally specified to indicate the system device handler should be automatically reloaded after the contents of the input files are loaded. [Note: Normally as many as 17 input files can be passed to the *slurp* loader; when /I is in effect the maximum number is lowered to 16 input files.]

If /V is specified, the standalone *slurp* loader is not used. Instead, a virtual *slurp* loader is used which generally takes longer to accomplish the same file loading. If the virtual *slurp* loader is in effect, the /I switch is ignored. [Note: On systems that do not support the standalone *slurp* loader, /V is always set by default.]

Glossary of terms used in this document [continued].

List of operating systems that support *slurp* binary format.

- 1) The R-L Monitor System [aka MS/8 when submitted to DECUS].
- 2) DIBOL-8 [actually a hacked-up version of the R-L Monitor System with the command names patched for consistency with COS-300/310 which replaced DIBOL-8].
- 3) POLY BASIC which supports various storage devices such as the TC01/TC08/TU55/TU56 DEctape, TC12 LINCtape, DF32 and RF08.
- 4) EDUSYSTEM 30 [DEC's commercial release of POLY BASIC with only modest modification].
- 5) EDUSYSTEM 15/30. This is an implementation of POLY BASIC specifically for the TD8E with a TU56H [single DEctape drive] on a 4K PDP-8/E with the MR8E-C ROM loaded into locations 77400-77777. Due to the presence of the ROM in extended memory, the M837 extended memory controller [or equivalent] is required; maximum memory allowed is 28K if other operating systems are used that can take advantage of extended memory such as P?S/8.
- 6) All releases of P?S/8. P?S/8 systems configured for DEctape can directly read R-L Monitor System DEctapes. *Slurp* format binary files can be transferred to P?S/8 DEctapes directly. P?S/8 configured for use on LINC-8 or PDP-12 LINCtapes can directly read R-L Monitor System DEctape images once converted to LINCtape format using the prevailing tape conversion programs [such as PRTC12F].

System Console

This is the main operator terminal of P?S/8. It is generally a device 03 [for input] and device 04 [for output] combined keyboard and [typically] low-speed teleprinter for all system messages and the output of most programs run under P?S/8.

Variations exist for the system console device such as display terminals [instead of hard-copy terminals] which are generally compatible with the original supported device, the Teletype Model 33 teletype [usually the Automatic Send/Receive or ASR variant] which allows low-speed paper-tape to be read or punched as required.

Glossary of terms used in this document [continued].

P?S/8 includes support for a variety of system console device variants such as the VT-100 [and compatible] family of display terminals as well as older models such as the VT-52 and the VT05. [Note: The VT05 requires the use of the M8655 KL8-JA serial interface card with the fill-jumper set unless used at 300 baud or less.]

Hard-copy terminals can be configured for minimal support of buffer character delete acknowledgment or the method defined in the TOPS-10 operating system for hard-copy terminals [which involves surrounding all deleted characters, printed in reverse order as they are deleted, by a leading and trailing `\` character].

The P?S/8 Logical Console Overlay supports several emulated terminals using hardware devices such as the VT8E and the PDP-12 VR-12/14 display oscilloscope. Certain support parameters can be tailored for use with these devices as required.

Because P?S/8 supports an absolute standard for the console device, all conforming programs need not be involved in the details of the support. Programs that are non-conforming may have limited compatibility with only certain console devices and/or machine models. [Note: Many older programs cannot be executed on DECmate systems unless the internal console routines are upgraded to full P?S/8 compliance. Example programs are available in other documents to illustrate conformance requirements.]

System kernel

The system kernel on virtually all PDP-8 operating systems refers to memory locations 07600-0777 possibly augmented by other memory areas depending on the operating system.

This hard system convention came about due to certain other software conventions that evolved over time.

Paper-tape loading environment.

The original paper-tape "operating system" addresses most of the issues.

The original PDP-8 three-cycle data break devices reserve locations 07750-07755; programming the related devices changes the contents of these locations out of necessity:

Glossary of terms used in this document [continued].

07750-07751 are used by either the DF32 or RF08 fixed-head disk systems [which are mutually exclusive]. Location 07750 is used to setup the word count for a DMA transfer; location 07751 is set to the desired transfer address [less one] as part of the basic programming of these devices.

07752-07753 serve the analogous functions as on the DF32/RF08 for the TC58 .5 inch magnetic tape controller.

07754-07755 serve the analogous functions for the TC01/TC08 DECTape controller.

These important memory locations cannot be used by typical PDP-8 programs which do not address these specific devices. As such, this area of memory is defined as off-limits to any general-purpose program.

The most fundamental usage of memory is the minimal loader function. The earliest PDP-8 loading program is the **RIM** loader designed to load into the rest of the page following the reserved three-cycle data break locations. The **RIM** loader occupies locations 07756-07776 and is started in 07756. In terms of what is practical to be loaded into this small memory area, the **RIM** loader represents the smallest possible memory usage.

Note: Some years later, a user program was developed known as the "help" loader which is slightly smaller, but must be transiently loaded elsewhere in memory to be of any practical use. It has a small user following and is generally used to load the **RIM** loader and perhaps the **BIN** loader using the shortest possible manual procedure to key in a paper-tape binary data loader. The "help" loader is not practical for loading in larger programs due to its inefficient use of paper-tape compared to the **RIM** and **BIN** loaders.

The **RIM** loader has many shortcomings which limits its usage to the most compromised situations such as when used with certain diagnostic programs designed to be capable of destroying all other locations in memory. Generally, the **RIM** loader is used to load the **BIN** loader into memory for further program loading purposes.

Glossary of terms used in this document [continued].

The **BIN** loader has several advantages over the **RIM** loader:

- 1) The **BIN** loader supports automatic incremental loading; The **RIM** loader requires each memory location to be preceded by the loading address. The **BIN** loader only requires a loading address when a non-adjacent memory area must be loaded.
- 2) The **BIN** loader supports a checksum at the end of the loading process. If the accumulator is clear, the loading can be presumed to have been successful; a non-zero accumulator indicates one or more errors occurred during the loading attempt.
- 3) while both the **RIM** and **BIN** loaders support loading into any memory field [the operator must set the data field [DF] switches on the computer front panel before starting the loading program]. only the **BIN** loader supports FIELD settings within the binary paper-tape contents. Programs can load into any memory field[s] as necessary.

The exact memory locations used by the **BIN** loader changed over time; multiple versions were released, each with slightly different memory usage. It is believed that certain memory ranges within the **BIN** loader were reserved to support some short-lived purpose of allowing locations in the last page of memory [07600-07777] to be available to certain diagnostic programs. With the release of additional **BIN** loader variants and the prevalence of storage-based operating systems, these conventions became obsolescent. With few exceptions, diagnostic programs do not reserve any memory locations in the range of 07600-07777 with the following exceptions:

- 1) The program may destroy all but 07756-07776 thus preserving the **RIM** loader as discussed above.
- 2) Certain programs may destroy certain locations in the range of 07600-07747 during program execution. Depending on the specific **BIN** loader used, this might be acceptable by design; the worst case is the **RIM** loader must be used for all further loading purposes.

with little exception, all standard DEC paper-tape programs obey these conventions.

Glossary of terms used in this document [continued].

Note: There are a few known "rogue" diagnostic programs that do not conform to these conventions. By not only assuming a specific implementation of the **BIN** loader, but also loading initial data into the reserved memory locations, these programs cannot be loaded by storage-based operating systems.

These programs would not have been released had certain DEC software managers been more competent. All known examples are PDP-8/E-specific updates to existing diagnostics for earlier models. The precursor programs conform to the conventions [by merely storing into the reserved locations and not directly loading initial data there].

New staff members were not properly trained regarding the exact nature of the conventions; without proper management, not only were these mistakes not corrected, they were released to DEC marketing. Many customers had problems with them due to their inherent incompatibility with prevailing DEC-supported storage-based operating systems [such as the Disk Monitor and OS/8].

P/S/8 supports a mechanism that can be used to remedy this situation in most cases; each program requires a slightly different solution. For more information, consult documentation of the P/S/8 BIN utility in other documents.

BIN loader startup convention.

Location 07777 is reserved as the universal startup location for all versions of the **BIN** loader. It is implemented as a JMP instruction to the internal starting location of the specific **BIN** loader implementation used.

BIN loader implementations can encompass loading into locations 07600-07747 depending on the specific version. In truth, nearly the entire highest page of memory [07600-07777] was eventually reserved in the sense that programs should avoid these locations. Storage system environments.

While there are several viable single-user storage-based operating systems for the PDP-8, all such systems reserve locations 07600-07777 with an exit to location 07600 to restart the operating system.

Glossary of terms used in this document [continued].

This includes the DECTape Library System, the Disk Monitor System, The R-L Monitor System, the Disk Monitor System, DIBOL-8, POLY BASIC, P?S/8, OS/8 [formerly PS/8], OS/78, OS/278, EDU-System 30, EDU-System 15/30, and COS-300/310.

Certain systems such as OS/8 and COS-300/310 require at least 8K memory [12K in certain configurations]. The system kernel uses x7600-x7777 where x is 1 [or 1 and 2 if 12K memory minimum is required]. Due to sloppy system conventions, all memory fields are considered to have x7600-x7777 reserved; user and system programs must avoid this area in all memory fields.

P?S/8 requires 4K minimum for most configurations; for certain configurations, a minimum of 8K is needed. However, the extended memory segment reserved is x6000-x7777 where x is the highest memory field available. All existent extended memory in any lower memory fields [assuming at least 12K is available] are available to system and user programs. This is especially useful for applications that require contiguous 15-bit addressing such as all programming for the FPP hardware and various software-based runtime systems that emulate a similar environment. [Systems such as OS/8 incorporate crude kludges to attempt to compensate for bad design when programs such as Fortran IV are run; had OS/8 been designed for a memory model similar to P?S/8, these kludges would be unnecessary.]

Most system kernels, with the exception of the paper-tape software and the DECTape Library System, provide some form of system service to conforming programs. [The DECTape Library System does have read/write routines callable from outside of the system kernel, but the functionality of this system is extremely limited; saving images of memory and loading saved images of memory are the primary functions; all other operations are involved with either the creation of a starter system or maintenance of the system itself.]

Glossary of terms used in this document [continued].

POLY BASIC, EDU-System 30 and EDU-System 15/30 are closed environments that only provide an environment to edit, compile and run BASIC programs.

The R-L Monitor System, as a raw implementation of an operating system, provides merely a framework for system programs. When a system program is run, a list of input files is passed in the system kernel along with a 12-bit value of the user-supplied =xxxx expression. The system does not provide any means to access the files, as the mechanism used to load the program image is merely a special-purpose loader for the image of the program; a system reboot routine is provided starting at 07600.

Since the R-L Monitor only runs on TC01/TC08/TU55/TU56 DECTape [only drive 0 is supported], each program image must contain its own internal DECTape read/write subroutine [typically patterned after the system device handler used internally by other parts of the operating system only].

Conforming programs work with the R-L framework to process input files. Where applicable, binary output is created to a dedicated output area defined for the purpose. Despite these limitations, PDP-8 assembly programming and several high-level language implementations are available.

Note: The R-L Monitor System is the precursor to P?S/8. All of the limitations present were overcome; the system kernel provided read/write capability directly; as such, P?S/8 is device-independent and runs on many configurations.

The OS/8 "family" of operating systems are mostly compatible with each other to within internal differences that are needed to run on specific models. There are confusing system names borne of a mix of marketing influences attempting to create a product without clear distinctions or an actual hardware requirement where the specific version determines exactly which models the system will run on.

Glossary of terms used in this document [continued].

Most notably, OS/78 Version 4 is designated for the DECmate I and OS/278 is designated for the DECmate II, III and III+ [Version 2 is the only viable version; Version 1 was never a complete release and is only found on diagnostic systems that weren't updated]. However, neither of these systems completely function. The console hardware of the DECmates is sufficiently incompatible with former machines that many released programs have bugs and certain problems cannot be solved due to inadequacies of the kernel; despite requiring a kernel twice the size of the equivalent of any of the above [or more] there is adequate memory space to fix these problems.

Additional incompatibilities of internal system calls exist due to sloppy system maintenance. A few programs, such as Kermit-12, navigate around known differences by using safe values on parameters such as maximum file size when opening a file. The program performs its own check for CPU model to better adapt to the differences of the DECmate environment [not provided by the system kernel].

All versions of OS/8 have general problems with the directory structure and badly-designed system date handling. As of this writing, OS/8 has no method for reckoning the current date for well over a decade. [Arguments can be made this capability actually ran out in 1978.]

The Disk Monitor System provides a convoluted series of system components which are oriented around the few system programs available. A read/write facility is available, but all operations are performed in an inefficient manner a single block at a time. Every other system discussed here supports the concept of multiple block read/write transfers; the performance on TC01/TC08/TU55/TU56 DECTape [one of the few configurations supported; the system cannot run on any other devices other than DF32 and RF08 due to incompatibility with newer hardware] is most notably poor on DECTape.

The P?S/8 system kernel provides device-independent efficient multiple-block system device handler calls for any conforming program. Various program loading modes are available ranging from a primitive rapid program load essentially an outgrowth of the R-L Monitor loading method through a full implementation of a P?S/8 system program.

Glossary of terms used in this document [continued].

[Note: P?S/8 system programs can also define overlay structures.]

The P?S/8 system kernel also provides file information that is an outgrowth of the R-L Monitor method of passing input files. Output files are also supported through a provided counter of output versus input files. [Up to 17 files can be passed with any number of input or output files.]

Program option switches are passed using the 36-bit model from TOPS-10 [implemented as three 12-bit words]. Any and all options in the range of /A through /Z and /0 through /9 can be passed in any execution-class command line.

The system date is available in memory [future releases are changing the format for easier date reckoning; the new date range is 01-Jan-1900 through 31-Dec-2411] to allow conforming programs to display the current date.

P?S/8 supports an optional memory-based component known as the Logical Console Overlay. Conforming programs change the physical console [and system line printer] handling to logical calls to the Overlay if present. A control word in the system kernel provides all information on interfacing with this component [when in effect].

Various additional control bits are implemented in the P?S/8 system kernel. A notable example is the presence of a logical abort bit. This feature allows proper operation on DECmates that lack compatible console hardware [which cannot pass back the information regarding program abort when exiting to the system kernel at 07600].

P?S/8 is under major development as of this writing. A new optional SHELL overlay will be implemented to allow an advanced directory structure and radically different command processor mechanism reminiscent of other operating systems. A control bit is available to allow conforming programs the ability to interface with the greatly expanded system communications and information structure of the SHELL environment. Additional system services work with information that is available either virtually or in memory depending on the specific configuration. Conforming programs have access to a variety of command-specific parameters; the scope of the available information far exceeds that of OS/8, yet the system can still be minimally functional on a 4K machine.

Glossary of terms used in this document [continued].

Standalone programs need not be concerned with the particulars of either the basic P?S/8 or SHELL environments; the only proviso is the program cannot be expected to run on DECmates. Fully conforming programs can run on any configuration of P?S/8 including all DECmates.

System Line Printer.

The system line printer is an optional component of any P?S/8 system, generally consisting of a high-speed hard-copy printing unit connected by a separate interface. The standard line printer is device 66 with a parallel interface that conforms to Centronics parallel and related standards as made by a variety of companies such as Data Products [both with and without DEC branding that changes little to no internal components]. [Note: DEC sold a Centronics printer with the DEC branding on the outside and a cooling fan removed from the inside; no other changes were made to this device. Informed users saved money purchasing the Centronics variant.]

P?S/8 universally supports the device 66 line printer as required. Other printers can be supported through customization of the Logical Console Overlay, which allows flexible support of several categories of device including serial printers that require reverse-channel flow-control support by the use of the Control-S/Control-Q industry-standard protocol. The device 66 printer support is generally enhanced by the use of software buffering which can produce dramatic improvement of printing throughput especially on higher speed models. [By way of comparison, OS/8 cannot drive a device 66 line printer beyond 300 lines per minute while P?S/8, with the Logical Console Overlay enabled, has successfully printed on suitable printers at over 1200 lines per minute.]

Note: Many older programs cannot support printers other than the device 66 standard models. By conforming with P?S/8 standards, any program can be made to work with all supported printers. [As in the case of the P?S/8 system console, example programs are available in separate documents to illustrate conformance requirements.]

Glossary of terms used in this document [continued].

TFS

The TFS [or Tiny File System] is the name of the primary user file system used in the R-L Monitor, DIBOL-8 and P?S/8. TFS files are fixed-length and always stored in 16 contiguous blocks.

Since the file structure is deliberately simplified, the entire TFS directory structure can be completely cached in the keyboard monitor and related utilities. This allows high-speed access to all TFS files regardless of physical storage position. The TFS directory is only updated when necessary; if a file slot is reused, there is no need to update the directory. If a file physically adjacent to a previously-loaded file requires loading into the editing buffer, there is no need to perform a physical directory access when changing files. This method of file access offers profound performance improvement on serial access devices such as DECTape and LINCTape over more conventional approaches.

Note: The R-L Monitor System and DIBOL-8 do not support logical device units; the only supported storage device is a single DECTape drive connected to a TC01/TC08 DECTape controller. The starting block of the TFS directory on these systems can be any storage block without restriction.

P?S/8 places a boundary restriction on the starting block number of the TFS directory; all files must fall on boundaries that are even multiples of eight blocks. This allows P?S/8 to supply file starting block and logical unit number [in the range of 0-7] in a single 12-bit word to any conforming system program.

TFS *slurp* binary format files from the R-L Monitor System and DIBOL-8 are fully compatible with P?S/8; however, text files must be converted using the R-L "FIXUP" program available from the developers of P?S/8. Once this is accomplished, all of the files can be transferred to P?S/8-based DECTapes to allow all P?S/8 system programs to use these files. The memory-resident commands of the keyboard monitor can directly access the files on DECTape after the FIXUP process has been completed. This allows a complete file migration to P?S/8 from these older systems; two DECTape drives are required for the file transfer.

[End of glossary of terms used in this document]

[End-of-file]